

Efficient Approach for Mining Frequent Subgraphs in Weighted Single Large Graphs

by

Nahian Ashraf

Exam Roll: Curzon Hall-259

Registration No: 2014-916-605

Session: 2014-15

Riddho Ridwanul Haque

Exam Roll: Curzon Hall-237

Registration No: 2014-416-606

Session: 2014-15

A project submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
UNIVERSITY OF DHAKA

January 6, 2019

Declaration

We, hereby, declare that the work presented in this project is the outcome of the investigation performed by us under the supervision of Dr. Chowdhury Farhan Ahmed, Professor, Department of Computer Science and Engineering, University of Dhaka. We also declare that no part of this project has been or is being submitted elsewhere for the award of any degree or diploma.

Countersigned

Signature

.....

(Dr. Chowdhury Farhan Ahmed)

Supervisor

.....

(Nahian Ashraf)

.....

(Riddho Ridwanul Haque)

Abstract

In the modern world, graphs are a necessary tool used to symbolize the complex inter-relationships between different entities in many real world applications. It is desirable to extract useful and interesting patterns from graph databases. However, interactions between different elements in real life are not equally important and algorithms that do not acknowledge this fact will fail to identify many influential patterns. The consideration of edge weights during frequent subgraph mining can help us discover more useful subgraphs compared to its unweighted counterparts. Although several recent literature have proposed weight adaptation in transactional graph mining databases, the consideration of edge-weights on single large graphs, remains, somewhat, unexplored. But the transactional approach is not always applicable specially in large networks where the aim is to find regularities in a single graph rather than a set of graphs. Our proposed algorithm resolves this problem by assigning weights to interactions represented by edges and then taking both the weight and frequency of a subgraph into account in the single large graph scenario. Considering the significance of the interactions between different types of entities, the algorithm successfully mines subgraphs whose weighted support is greater than a given user-defined threshold, thus outputting subgraphs which are both significant and frequent from the application's perspective. In addition, it is equipped with various prunings and optimizations to make the mining process faster and provide greater scalability. Ample experimental analysis conducted on large real-life datasets evaluate the performance of the algorithm and the results demonstrate its superiority over existing single large graph mining algorithms in terms of efficiency, quality and scalability.

Acknowledgements

All praise is to the Almighty, who is the most gracious and most merciful. There is no power and no strength except with Him.

Our deep gratitude goes to our thesis supervisor, Dr. Chowdhury Farhan Ahmed, Professor, Department of Computer Science and Engineering, University of Dhaka, for his proper guidance in our research field. He has shared his expert knowledge gathered from working in the field over an extensive period, and has been an integral support in our thesis work by constantly keeping updates and urging us to do something significant.

We are extremely grateful to Md. Ashraful Islam for his constant guidance and support.

We want to thank our families and friends for their unwavering love and support. The opportunities that our parents have made possible for us determines the personalities we have built and the work that we produce today.

Lastly, we want to thank the Department of Computer Science and Engineering, University of Dhaka, its faculty, staff, and all other individuals related to the department. The department has facilitated us throughout our undergraduate program and subsequent thesis, and has also formed the base for our future endeavours.

Nahian Ashraf
Riddho Ridwanul Haque
January, 2019

Contents

Declaration of Authorship	i
Abstract	ii
Acknowledgements	iii
List of Figures	vii
List of Tables	ix
1 Introduction	1
1.1 Problem Definition	5
1.2 Motivation	6
1.2.1 Motivating Example	7
1.3 State of the art	8
1.4 Challenges that remain	10
1.5 Objectives	11
1.6 Contributions	11
1.7 Outline of the Report	13
2 Related Work and Preliminary Concepts	14
2.1 Frequent Pattern Mining	14
2.2 Weighted Frequent Pattern Mining	16
2.3 Graph Mining	18
2.3.1 Definitions	18
2.3.1.1 Single Large Graph	19
2.3.1.2 Weighted Single Large Graphs	19
2.3.1.3 Subgraphs	20
2.3.1.4 Graph Isomorphism	21
2.3.1.5 Subgraph Isomorphism	22

2.3.1.6	Minimum Image Based support Of A Subgraph . .	23
2.3.2	gSpan	25
2.3.3	GraMi	25
2.4	Existing Work and their Limitations	26
2.4.1	WIGM	26
2.4.2	ReSuM	28
2.4.3	EdWePat	30
3	The Proposed Approach	32
3.1	Preliminary Concepts	32
3.1.1	Problem Statement	32
3.1.2	Research Challenges	34
3.2	Algorithmic Details	37
3.2.1	Outline of WeFreS	37
3.2.2	Determining an Upper Bound of Maximum Possible Weight	39
3.2.2.1	Calculation of <i>MaxPosW</i>	39
3.2.2.2	Taking Advange of Unimodality	48
3.2.2.3	Maintaining a Sorted List of Remaining Edges . . .	52
3.2.3	Determining a Lower Bound of the MNI of a Subgraph . . .	60
3.2.3.1	Identifying a Weighted Frequent Subgraph	60
3.2.3.2	Identifying the Extendability of a Subgraph	61
3.2.4	The CSP Model	62
3.2.4.1	Using a Status Flag	64
3.2.5	Decision Conditions	68
3.2.6	Deletion of Infertile Node and Edge Labels	70
3.2.7	Complete Design of WeFreS	71
3.2.7.1	Initiating the Algorithm	71
3.2.7.2	Subgraph Extension	73
3.3	Time Complexity of WeFreS	74
4	Experimental Results and Analysis	76
4.1	Data Description	76
4.1.1	Weight Inclusion	78
4.2	Environmental Setup	79
4.3	Performance Evaluation	79
4.3.1	Comparison with Weighted Frequent Subgraph Mining Ap- proaches	80
4.3.1.1	Comparison with Baseline Algorithms	80
4.3.1.2	Comparison with ReSuM	85
4.3.2	Comparison with Unweighted Frequent Subgraph Mining Approaches	91
4.4	Feasibility Analysis	95

5	Conclusions	97
5.1	Research Summary	97
5.2	Future Research Scopes	98
	Bibliography	105
	List of Notations	105

List of Figures

1.1	A Sample Node and Edge-labelled Graph	4
2.1	An Example of an Edge-Weighted Subgraph	20
2.2	An Example Graph	27
2.3	A Test Subgraph	27
3.1	A Sample Graph	33
3.2	Output from the Sample Graph with Threshold 30	34
3.3	Absence of Downward Closure Property	35
3.4	A Sample Subgraph	40
3.5	A Bar Chart Representing the Unimodality of h_i	46
3.6	Demonstration of Ternary Search on h_i (i)	50
3.7	Demonstration of Ternary Search on h_i (ii)	51
3.8	Demonstration of Ternary Search on h_i (iii)	52
3.9	Initialization of the Segment Tree for the Graph in Figure 3.1	53
3.10	The Initial Segment Tree for the Graph in Figure 3.1	54
3.11	An Initial Edge Added to Create a Subgraph	54
3.12	Deletion: The Segment Tree for the Graph in Figure 3.1 after the Creation of the Subgraph in Figure 3.11.	55
3.13	The Segment Tree for the Graph 3.1 after Subgraph 3.4	56
3.14	Subgraph Constructed after Removing the Edge in Figure 3.11 from the Subgraph in Figure 3.4	57
3.15	Insertion: The Segment Tree after Removing the Edge in Figure 3.11 from the Subgraph in Figure 3.4.	58
3.16	The Segment Tree for the Graph in Figure 3.1 after Creating the Subgraph in Figure 3.4	59
3.17	Using a Status Flag with the CSP Model	65
4.1	Runtime Comparisons with Baseline Algorithms	81
4.2	Number of Frequent Subgraphs Mined in a Given Time Constraint	83
4.3	Search Space Comparison with Baseline Algorithms	84
4.4	Runtime Comparison with <i>ReSuM</i> in the <i>Amazon</i> Dataset	86
4.5	Number of Subgraphs Mined in a Given Time Constraint (<i>Amazon</i> Dataset)	87

4.6	Search Space Comparison with <i>ReSuM</i> in the <i>Amazon</i> Dataset . . .	89
4.7	Memory Usage Comparison with <i>ReSuM</i> in the <i>Amazon</i> Dataset . .	90
4.8	<i>MiCo</i> -unweighted: Runtime Comparison with <i>GraMi</i>	92
4.9	<i>Amazon</i> -unweighted: Runtime Comparison with <i>GraMi</i>	93
4.10	<i>FreeAssoc</i> -unweighted: Runtime Comparison with <i>GraMi</i>	93
4.11	<i>MiCo</i> -unweighted: Memory Usage Comparison with <i>GraMi</i>	94
4.12	<i>FreeAssoc</i> -unweighted: Memory Usage Comparison with <i>GraMi</i> . .	94
4.13	<i>Amazon</i> -unweighted: Memory Comparison with <i>GraMi</i>	95

List of Tables

2.1	A Transactional Database	15
2.2	A Weight Table	16
2.3	An Example of Graph Isomorphism	22
2.4	An Example of Subgraph Isomorphism	23
2.5	An Example of Minimum Image Based Support (MNI)	24
3.1	Assignment of Edge-Weights in the Example Graph	34
3.2	Example List of Remaining Edges	40
3.3	Example Use of Lemma 3.1 to Extend the Subgraph in Figure 3.4 .	42
3.4	Unimodality of h_i in the Example Subgraph	45
3.5	Valid Mappings of the Subgraph in Figure 3.14	64
4.1	Dataset Specifications	77
4.2	Weight Distributions and Specifications	79
4.3	Memory Usage Comparisons	85

List of Algorithms

1	FINDSUBGRAPHS	37
2	SUBGRAPHMINER	38
3	TERNARYSEARCH	49
4	QUERY	59
5	MNILOWERBOUND	67
6	GETMNISTATUS	69
7	DELETEINFERTILELABELS	71
8	FREQUENTWEIGHTEDSUBGRAPHMINING	72
9	SUBGRAPHEXTENSION	73

Chapter 1

Introduction

Data from various fields around the globe are influenced by a collection of many real-world events and factors. In many cases, such data holds the key to discovering many hidden patterns in real-world scenarios which were previously unknown to us. The correct interpretation and analysis of data can thus lead us to gaining useful insights and having a better understanding of how things work. The knowledge that can be extracted from it can be used to take better and more informed decisions. As such, data is an invaluable resource whose utilization is dependent on our ability to extract information from it. But with data stores around the world continuously growing in size and becoming more robust, knowledge extraction from data via manual observation no longer remains feasible. From the context of this problem, the field of data mining emerged.

Data mining is the process of extracting useful knowledge from large databases. In other words, it is the act of ‘mining’ a large database to extract useful chunks of information from it [4][27]. Fundamental to this quest of Knowledge Discovery from Data (often abbreviated as KDD) is the problem of mining frequently appearing patterns in the dataset.

Frequent pattern mining concerns itself with identifying patterns that appear frequently in a large dataset using efficient and scalable approaches. It is an important domain of data mining whose significance is proven by its application in a wide variety of problems including clustering [28], correlation analysis [4], classification [29], recommendation [30] and outlier detection [26][32]. Frequent pattern mining helps us extract features from the data that is available to us, and is regularly used as an intermediate step by many algorithms to interpret the data in terms of the patterns found in it [31].

Early research on frequent pattern mining focused on mining frequent itemsets from transactional databases[5][6]. Such databases are seen as a collection of transactions. Transactions are in turn seen as a collection of items, that are found together in a real-life event. As an example, let us consider a super shop that maintains a list of all items bought by an individual customer during a single visit to the shop. Each such collection of items which have been bought together can be viewed as a transaction. For example, if a customer X goes to the shop to buy milk, cereals, a light bulb and a DVD player, the set {milk, cereals, light bulb, DVD player} can be seen as a transaction. During analysis of such a database, frequent itemset mining algorithms can be used to identify items that frequently appear together. This knowledge may be used by the super shop owner to offer special discounts for items being bought together in order to increase sales.

A key weakness of traditional frequent pattern mining approaches that do not take weights of items into account, is that they consider the significance of all items appearing in a transaction to be equal. However, that may not always be desirable. Referring to the super shop example described above, the event of a customer buying a cereal box may be far less significant to the owner than the sale of a household blender. In real life scenarios, the sale of cereals may be far more common, but users would want to mine itemsets which contain information about

items that are usually bought along with a blender, even if they are less frequent than itemsets containing cereal boxes and eggs.

To address this problem, approaches such as WFIM[1] were developed, where the aim is to mine weighted frequent itemsets. To explain this further, the frequency of each itemset is multiplied by the average of the weights of the items in it to calculate its weighted support. WFIM mines frequent itemsets having high weighted support values. Referring again to the super shop example, if blenders are assigned much higher weight values than cereals, it is likely that the itemsets containing blenders shall be mined in spite of their relatively lower frequencies. In general, consideration of weights can ensure that more useful itemsets are mined from the database.

However, not all datasets available around the world today contain transactions that may be represented as itemsets. The itemset representation ignores any and all relations and connections that may exist between the items, other than the fact that they appear in a transaction together. In many cases, the items or entities present in a dataset are linked with one another, combining together to form complex networks [2][17][19][25]. As an example, let us suppose a social networking site which is used by millions of users. Each user may be ‘friends’ with a number of other users in the site. Each ‘friendship’ can be represented as a link between two users. Collectively, the users and the links form a complex network that can be best represented by a graph.

To elaborate on the aforementioned point, when representing the social network in the form of a graph, each user can be represented by a node. A friendship between two users can be represented by an edge connecting the nodes corresponding to the two users.

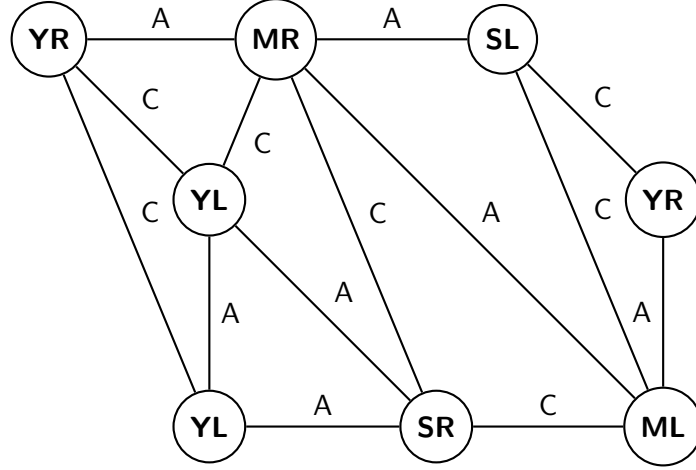


FIGURE 1.1: A Sample Node and Edge-labelled Graph

Based on any relevant criteria, we may chose to classify each user into belonging to a certain type of category. Suppose users are classified based on their age (e.g. young, middle-aged, senior) and their political orientation (e.g. right wing/left wing). Such a classification may divide the users into at most six different categories which we may simply denote as YR, YL, MR, ML, SR and SL. We may also classify the proximity of two users with one another by partitioning all friendships into two discrete classes, close and acquainted (which we can simply denote by C and A). Such metadata associated with the users and their relationships can be represented by a graph using node-labels and edge-labels. A small section of the social network when represented by a graph may appear as in Figure 1.1.

Knowledge discovery from graphs, or graph mining is one of the most challenging and important domains of data mining, posing a wide variety of research challenges[7]. In this work, we tackle the important problem of weighted frequent subgraph mining from a weighted single large graph. While efficient and effective approaches exist for mining frequent subgraphs from an unweighted single large graph [8], the consideration of weights is essential for many applications [21][33]. Weighted frequent subgraph mining from single large graphs, however, remains a

largely unsolved problem. The absence of the *Apriori* property [5] during weighted subgraph mining which is explained in section 3.1.2 adds an additional layer of difficulty in this context.

In this work, we propose **WeFreS**, an efficient algorithm that can mine weighted frequent subgraphs from a weighted single large graph. **WeFreS** overcomes the challenges of the absence of the *apriori* property by proposing a novel approach for determining the maximum possible weight a subgraph can attain by being extended by one or more edges. We further modify the existing CSP model[8] for finding a lower bound of the *MNI*[9] of a subgraph, and adapt it to fit in to a weighted framework. Finally, we demonstrate the efficiency and effectiveness of **WeFreS** by virtue of experimental results gained from real-world datasets.

1.1 Problem Definition

Our proposed approach **WeFreS** mines all subgraphs of an edge-weighted single large graph whose weighted support satisfies a given user defined threshold.

Formally, given a weighted single large graph G and a user-defined threshold τ , **WeFreS** outputs all subgraph patterns whose weighted support is greater than or equal to τ .

Formal definitions and explanations of the terms used in the above statement are elaborated in section 2.3.1. In this introductory section, we restrict ourselves to only giving an intuitive outline of the problem.

A single large graph is a large network, where the entire dataset can be represented as an individual graph[8]. In the scope of this work, we are concerned with

edge-weighted single large graphs where the weight of each edge is provided and the weight of an edge is a function of the labels of the nodes adjacent to the edge, and the label of the edge itself. That is, all edges in the graph connecting nodes having the same pair of labels, and having the same edge label will have equal weights. In Figure 1.1, there are two edges labelled ‘A’ connecting nodes labelled ‘YL’ and ‘SR’. Both these edges shall have the same weight values, while edges connecting a different set of node and edge labels can have different weights.

Similar to WFIM[1], we define the weighted support of a subgraph as the weight of a subgraph multiplied by its frequency. The weight of a subgraph is the average of the weights of all edges present in it. The frequency of a subgraph is measured in terms of its MNI index[9] in the large graph dataset, a measure that has been used as the metric for defining the frequency of a subgraph in many recent single large graph mining approaches [8] [12]. Details regarding the MNI index are given in section 2.3.1.6.

WeFreS takes a user-defined minimum weighted support threshold value τ as input and outputs all subgraphs having weighted support greater than or equal to τ . The results are both sound and complete.

1.2 Motivation

Single large graphs are often essential for representing data from different real-world applications such as social networks in social network analysis [17], protein-protein interactions in bioinformatics [34], citation and collaboration graphs among authors from different academic fields [35], financial transactions [33], chemical networks in cheminformatics [13], etc. The addition of weights on the edges of such graphs helps users specify the types of links that are more significant to them with respect to others. The higher the weight of an edge, the higher is its significance. Unweighted subgraph mining approaches [8] that work on single large graph do

not take weight into consideration while mining frequent subgraph patterns. Thus they tend to mine subgraphs with relatively low significances [12]. On the other hand, subgraph patterns that occur less frequently but are nevertheless significant due to the presence of edges with high weights in them, may not be mined at all, unless the thresholds are set to very low values. Setting the threshold to extremely low values may result in unacceptably long runtimes, and the mining of too many insignificant subgraphs, along with the significant ones.

Thus there exists the need for designing an efficient approach for mining weighted frequent subgraphs from single large graphs, which considers both the weight (i.e. significance of the subgraph from the application's perspective) and the frequency of a subgraph in a large graph. As a result, the subgraph patterns that are mined are ensured to be both significant and commonly found throughout the dataset. Thus, they can be used to find pattern-centric insights into the nature of the graph data, and can be used in applications including opinion mining [18], gene classification [19], image processing [14], software bug detection [15], text classification [16], graph anomaly detection [36] and spam detection [2].

1.2.1 Motivating Example

As an example of a potential application of identifying weighted frequent subgraphs from a large graph, let us consider the dataset in [21]. The dataset contains anonymous information from *Tagged.com* and contains information of over 5.6 million users, and information on interactions between them. The users can all be represented as nodes, and their interactions may be seen as edges. The users are labelled in terms of their age, gender and on whether they have been identified as spammers or not. These metadata on the users may be used to label the node corresponding to each user.

In order to study the nature of spam dispersion across a social network, one may wish to identify the frequent subgraph patterns through which spammers connect and interact with other users in the network, and identify the communities that are developed surrounding spammers in social networks.

In effect, we need an algorithm that will mine us frequent subgraph patterns involving spammers. However, the number of spammers in the network is much lower than the number of non-spammers. As such, subgraph patterns involving only non-spammers are much more frequent. Such patterns shall be mined by conventional frequent pattern mining algorithms ignoring the important subgraph patterns involving spammers in the network.

Weighted frequent subgraph mining can be used to solve this problem. By assigning high weight values to all edges denoting interactions with a spammer, and low weight values to other edges, we can assign high weight values to subgraphs containing interactions of spammers with other users. As such, these subgraphs shall be mined before other subgraphs in spite of their lower frequencies, owing to their high weight values.

Thus, there exists the necessity for designing an efficient weighted frequent subgraph mining algorithm that takes both the weight and the frequency of a subgraph into consideration, and outputs those containing high values of weighted support.

1.3 State of the art

Traditionally, research on graph mining has concentrated on two distinct subdomains, transactional graph database mining [7] and single large graph mining [8]. The former is concerned with mining graph databases, a collection of independent and relatively small graphs, each of which is seen as a transaction. The frequency

of a subgraph is the number of graphs in the database that contain at least one valid subgraph isomorphism with it. Thus, how many times a subgraph pattern appears in each graph is not a relevant concern of transactional graph database mining, we only need to determine if at least one valid subgraph isomorphism exists.

Thus, single large graph mining poses a different set of challenges. GraMi [8], proposed by Elseidy et al., is a state-of-the art approach for mining frequent subgraphs from single large graphs. However, GraMi does not consider edge-weights during frequent subgraph mining. The inclusion of weights to frequent subgraph mining, however, is a non-trivial problem, fundamentally due to the fact that weighted patterns do not respect the *Apriori* property, a fact that is discussed in detail in section 3.1.2. In almost all unweighted frequent pattern mining approaches, the *apriori* property is an essential tool for achieving a bounded search.

WIGM [11], proposed by Yang et al., proposes a bottom up approach for mining subgraph patterns that have high weight values, and does not consider the frequencies of the subgraphs that are mined. As such, there is no way to ascertain if the subgraph patterns mined by WIGM are regular occurrences or outlier patterns or graph anomalies.

ReSuM [12], proposed by Preti et al., proposes an approach for filtering the subgraphs mined by GraMi, to output subgraphs whose weight and frequency are both higher than the given user-defined threshold. As a result, ReSuM works as a filtration method, and can remove insignificant subgraphs that are mined by GraMi, but offers no solution for mining relevant subgraphs which are not mined by unweighted subgraph mining approaches as demonstrated in section 1.2.1.

To the best of our knowledge, while several solutions have been proposed for

weighted subgraph mining from transactional graph databases [22][37][38], there is no other existing approach that tackles the problem of mining weighted frequent subgraphs from a single large graph.

1.4 Challenges that remain

The amount of research done on weighted frequent subgraph mining from weighted single large graphs remains vastly limited, and none of the approaches mentioned in section 1.3 offer any counter-measure for adapting to the absence of the *Apriori* property during weighted subgraph mining. The absence of this property is explored in detail in section 3.1.2. Here, we simply mention that the absence of this property which means that a subgraph pattern which is not weighted frequent itself can be extended by edges of higher weight values to potentially gain a weighted frequent subgraph pattern. Thus, even if we test a subgraph pattern and determine that it is not weighted frequent, we cannot necessarily prune it and all of its extensions from the search space, as is done by unweighted algorithms. Attaining a bounded search is thus highly difficult.

Furthermore, recent frequent subgraph mining approaches working on single large graphs define a CSP (Constraint Satisfaction Problem) model that determines if the MNI support of a subgraph is at least equal to the given user-defined threshold [8][12]. However, as we shall discuss elaborately in Chapter 3, this approach in itself is inadequate for weighted frequent subgraph mining. In single large graphs, correctly determining a lower bound of the MNI support of a subgraph pattern remains a computational bottleneck. As such, redesigning the prevalent CSP model to fit into a weighted framework, and optimizing it as much as possible is of utmost importance in preserving the time practicality of our approach.

The key challenges that remain can thus be summarized as follows:

- Achieving a bounded and efficient search despite the absence of the *apriori* property during weighted frequent subgraph mining from single large graphs.
- Reformulating the prevalent CSP (Constraint Satisfaction Problem) model used in unweighted approaches to adapt it for use in a weighted framework.

1.5 Objectives

The objectives of our work were as follows:

- To design an efficient approach for mining subgraphs that are weighted frequent.
- To ensure the scalability and time practicality of our solution through necessary optimization and pruning techniques.
- To introduce an effective counter-measure to compensate for the absence of the *Apriori* property in weighted pattern mining.
- To reformulate the existing CSP model to use it in a weighted framework.
- To ensure that our algorithm shows similar performance to existing unweighted frequent pattern mining approaches when working on unweighted single large graphs or graphs where all edges have equal weights.

1.6 Contributions

Our proposed approach, **Weighted Frequent Subgraph miner(WeFreS)** overcomes the challenges mentioned in section 1.4 in an efficient manner. We introduce a novel technique to determine the **Maximum Possible Weight (*MaxPosW*)** a

subgraph can attain after being extended by one or more edges from the given input graph. The *MaxPosW* measure provides a much tighter upper bound for this value than the prevalent *MaxW* measure proposed in WFIM for weighted frequent itemset mining. Thus, it is better equipped to be used as an effective admissible heuristic to compensate for the absence of the downward closure property during weighted pattern mining.

Furthermore, we reformulate the CSP model used in recent unweighted subgraph mining approaches and adapt it for use in the context of weighted frequent subgraph mining in a weighted single large graph.

WeFreS proves to be highly efficient and experimental results gained from several real-world datasets show that it runs significantly faster than the baseline algorithms we compare it with as well as other previously proposed approaches. Hence, the *key contributions* of our work are as follows:

- Identifying and proving the unimodality of the distribution of maximum possible weight attainable by a subgraph after being extended by exactly k edges over k .
- Using the unimodal distribution to propose an efficient way to calculate a tighter upper bound of the maximum possible weighted support any subgraph can attain after being extended by one or more edges.
- Reformulation of the existing CSP model to fit it into our weighted framework.
- Proposing **WeFreS**, an efficient and scalable weighted frequent subgraph mining algorithm that works on edge-weighted single large graphs.
- Analyzing the performance of **WeFreS** using results gained from experiments run on several real-world datasets.

1.7 Outline of the Report

The remainder of this report is organized as follows. Chapter 2 introduces the preliminary concepts and contains an overview of various related works, exploring their strengths and weaknesses. Chapter 3 contains a detailed description of our proposed approach. Chapter 4 contains details and results of the experimental analysis that we have done. Chapter 5 concludes the report by providing a summary of our research endeavours and future research scopes implicated by our work.

Chapter 2

Related Work and Preliminary Concepts

In this chapter, we introduce the basic concepts of frequent pattern mining, weighted pattern mining and graph mining. Then we examine the existing subgraph mining algorithms related to our work and their limitations to highlight the necessity of designing an efficient algorithm to mine weighted frequent subgraphs in weighted single large graphs.

2.1 Frequent Pattern Mining

Frequent patterns are patterns that frequently occur in a database. Frequent patterns can be itemsets, subsequences, or substructures. These patterns highlight the recurring relationships in a given data set. The discovery of such patterns to understand associations and correlations between item sets in transactional and relational databases is known as frequent pattern mining.

Minimum support, min_sup is the threshold value used to identify interesting

patterns in a database. Support of a pattern is described as the number of transactions in which it appears in a database.

Thus, frequent pattern mining is the task of discovering all patterns P in a database D with transactions $T_1, T_2, \dots, T_n$ where P appears in min_sup or more transactions.

An example transactional database is provided in Table 2.1. If min_sup is set

TABLE 2.1: A Transactional Database

Transaction ID	Items
T_1	milk, bread, rice
T_2	milk, bread, pen
T_3	gold ring, pen
T_4	rice, diaper
T_5	milk, bread, detergent

to 2, a set of items, such as milk and bread, that appears thrice in the dataset (in T_1, T_2 and T_5), which is more than the min_sup , is a frequent itemset and a set of items like milk and detergent is infrequent.

Frequent Pattern Mining has for long been one of the leading concerns of Data Mining, with initial research leading to algorithms such as Apriori[5] and Frequent Pattern Growth[6], which focus on the mining of frequent unordered itemsets. The Apriori algorithm identifies the apriori or downward closure property, stating : All k -length subsets of a $(k+1)$ length frequent pattern must be frequent as well. This property plays an important role in all other forms of frequent pattern mining. A major weak point of the algorithm is the potentially large size of the candidate set that is generated at each step which is slow and inefficient.

Frequent pattern growth or FP-growth[6] is an improvement designed to eliminate the number of candidates generated by apriori[5]. The algorithm adapts

a divide-and-conquer strategy. It is one of the fastest and most influential approaches discovered on frequent pattern mining, with many algorithms working on different types of patterns employing the Frequent Pattern Growth approach.

2.2 Weighted Frequent Pattern Mining

Weighted frequent pattern mining helps us mine more interesting patterns by considering the relative significance of each data element in a database. In weighted pattern mining, weights of data elements are assigned according to the quantitative value and importance of that element in real world applications.

Contradictory to traditional pattern mining where all data elements are treated as equally significant, weighted pattern mining has weights $W_1, W_2, \dots, W_m$ assigned to each of the data elements. The weighted support, WS is the product of the average of the weights of the elements in the pattern and its occurrence or support in database D . In weighted pattern mining, a pattern P is frequent when the weighted support, WS of P is greater than a given threshold, min_sup .

Following table shows an example scenario where weights are assigned to the items in Table 2.1:

TABLE 2.2: A Weight Table

Item	Weight
milk	4
bread	1
rice	3
pen	2
gold ring	10
detergent	4

So, to calculate the significance of a pattern, the average of the weight assigned to each element is taken into consideration. For example, the weight of the pattern {gold ring, pen} from Table 2.2 is: $\frac{10 + 2}{2} = 6$

In Table 2.1, the pattern in question appears only once. Therefore, the support or occurrence of the pattern = 1.

So, the weighted support of the pattern,

$$WS(\{\text{gold ring, pen}\}) = 6 \times 1 = 6$$

The challenge in weighted pattern mining is maintaining the downward closure property or *Apriori* property illustrated in section 3.1.2. For example, if we set the value of min_sup to 5, although the itemset {gold ring, pen} will be weighted frequent, the itemset {pen} will not be so. The weight of a pen being 2, and with it appearing only in transactions T_2 and T_3 , the weighted support of the itemset {pen} shall be $2 \times 2 = 4$, which is less than the value of min_sup.

WFIM: Weighted Frequent Itemset Mining [1] reduces the number of frequent itemsets mined by setting a weight range and a minimum weight, allowing the user to balance support and weight of itemsets. While the support value of an itemset maintains downward closure property, the same cannot be said for weight values. Adding one item with high weight value to a weighted infrequent pattern can result in a weighted frequent pattern. This challenge is further explained by an example with subgraphs as patterns in section 3.1.2.

WFIM uses the concept of maximum weight for a transactional database and minimum weight for conditional database. The maximum weight ($MaxW$) and

the minimum weight ($MinW$) is defined respectively as the maximum and minimum values among the weight of all items. Using these concepts, FP growth algorithm is followed for WFIM with additional pruning conditions.

Even though the introduction of constraints like $MaxW$ and $MinW$ was an important step, the growing field of weighted pattern mining requires tighter bounds for pruning techniques to ensure efficiency.

2.3 Graph Mining

The representational power of graphs has made graph mining a dominant branch of data mining. Graph mining is the extraction of interesting patterns in the form of subgraphs from a single large graph or collection of graphs. A subgraph is frequent if its support in a given dataset is greater or equal to a minimum support threshold.

We will now discuss some definitions related to our work in the field of graph mining. We will then highlight two graph mining algorithms, namely gSpan [7] (Graph-based Substructure Pattern Mining) and GraMi [8] (Frequent Subgraph and Pattern Mining in a Single Large Graph). Several concepts introduced by these algorithms are used by our algorithm, **WeFreS**.

2.3.1 Definitions

In this section, we will discuss some definitions related to graph mining.

2.3.1.1 Single Large Graph

A single large graph is described by the four tuple (V, E, L, l) , where V represents a set of vertices, E represents the set of edges connecting the vertices, such that each edge $e \in E$ can be represented by (u, v) , where $u, v \in V$ and e connects nodes u and v . L represents the set of node labels, and $l: V \rightarrow L$ is a function that maps each node to a certain label. At times a single large graph is also represented by the six tuple (V, E, L, l, eL, el) , where eL denotes the set of edge labels and $el: E \rightarrow eL$ is a function that maps each edge to a certain edge label.

2.3.1.2 Weighted Single Large Graphs

A weighted single large graph is a single large graph where relative significance of nodes or edges connecting the nodes are provided.

A weighted single large graph can be described by the five tuple (V, E, L, l, w) . Edge weights in a graph determine the significance of each edge from the application's perspective.

In our work, we have considered the edge-weights to be a function of the labels of the nodes the edge connects. This allows users to relatively scale the importance of relations that are existent between nodes of different types. Thus, the function $w: (L \times L) \rightarrow R$ defines the relative significance of edge connections between nodes of different pairs of labels. As such, the weight of an edge (u, v) is defined by $w(l(u), l(v))$.

In the presence of edge labels, a weighted single large graph can be represented by the seven tuple (V, E, L, l, eL, el, w) , where the function $w: (L \times L \times eL) \rightarrow R$ defines the weight of each edge.

2.3.1.3 Subgraphs

A graph $S(V', E', L', l', w)$ is said to be a subgraph of graph $G(V, E, L, l, w)$, if $V' \subset V$, $E' \subset E$, $\forall v' \in V$, $l'(v')=l(v')$ and $(u', v') \in E' \implies (u', v') \in E$.

In simple terms, any graph that can be produced by deleting any number of nodes and edges from a large graph G is a subgraph of G . Figure 2.1 illustrates an example of a subgraph.

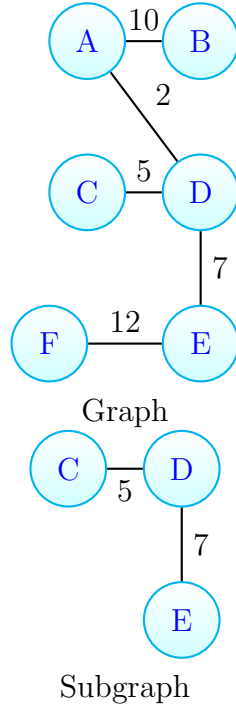


FIGURE 2.1: An Example of an Edge-Weighted Subgraph

In the example in Figure 2.1, the subgraph shown is formed by removing nodes A, B and F and the edges associated with these three nodes from the given graph.

The weight of a subgraph $S(VS, ES, LS, Is, ws)$, denoted by $sW(S)$ is defined as the average of the weights of all the edges present in the subgraph.

Formally,

$$sW(S) = \frac{\sum (w(l(u), l(v)) : (u, v) \in ES)}{|ES|}$$

The support of a subgraph denoted by sG is defined as the MNI of the subgraph which is explained further in section 2.3.1.6.

So, in the example in figure 2.1, the weight of the subgraph, $sW(S) = \frac{5+7}{2} = 6$

We define the weighted support of a subgraph S , denoted by $WS(S)$, as the product of the support of S and its weight. Formally,

$$WS(S) = sW(S) \times sG(S)$$

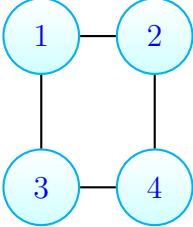
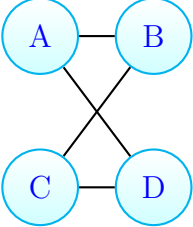
As such, the weighted support considers both the frequency of the subgraph in the main graph, represented by a support measure and the significance of the edges in it, represented by its weight.

2.3.1.4 Graph Isomorphism

Two graphs $G_1 (V, E, L, l, w)$ and $G_2 (V', E', L', l', w)$ are said to be isomorphic if there exists a function $f : V' \rightarrow V$, such that f is a bijection and f preserves adjacency of vertices that is $(u, v) \in G_1 \implies (f(u), f(v)) \in G_2$.

An example of isomorphic graphs are presented in Table 2.3.

TABLE 2.3: An Example of Graph Isomorphism

G_1	G_2	An Isomorphism Between G_1 and G_2
		$f(1) = A$ $f(2) = B$ $f(3) = D$ $f(4) = C$

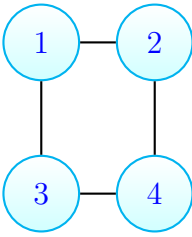
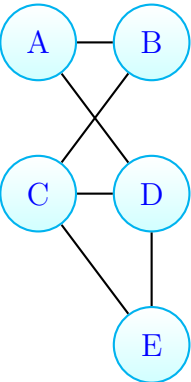
2.3.1.5 Subgraph Isomorphism

A graph $G' (V', E', L', l', w)$ is said to be subgraph isomorphic to a large graph $G (V, E, L, l, w)$ if there exists a function $f: V' \rightarrow V$, such that $l(f(V')) = l'(V')$ and $(u', v') \in E' \implies (f(u'), f(v')) \in E$.

It is to be noted that, the weight function applied on the edges of G and G' must be the same.

An example of subgraph isomorphism is shown in Table 2.4.

TABLE 2.4: An Example of Subgraph Isomorphism

G'	G	An Isomorphism Between Subgraph G' to G
		$f(1) = A$ $f(2) = B$ $f(3) = D$ $f(4) = C$

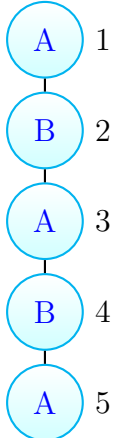
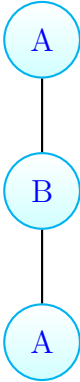
2.3.1.6 Minimum Image Based support Of A Subgraph

The Minimum Image Based support (MNI) is a widely used metric for measuring the support of a subgraph. MNI index indicates the minimum value from the number of times that a node in the target pattern is mapped to a distinct node in the graph. The metric, proposed by *Bjorn Bringmann and Siegfried Nijssen*, [9] is defined as follows:

“Let $f_1, \dots, f_m$ be the set of isomorphisms of a subgraph $S(VS, ES, LS, ls, ws)$ in a graph G . Also let $F(v) = \{f_1(v), \dots, f_m(v)\}$ be the set that contains the (distinct) nodes in G whose functions $f_1, \dots, f_m$ map a node $v \in VS$. The minimum image based support (MNI) of S in G , denoted by $sG(S)$, is defined as: $sG(S) = \min\{t \mid t = |F(v)| \text{ for all } v \in VS\}$.”

An example of this metric is explained in Table 2.5.

TABLE 2.5: An Example of Minimum Image Based Support (MNI)

Example Graph	Target Subgraph	Embeddings																											
		<table><tr><td></td><td></td><td></td><td></td></tr><tr><td></td><td>1</td><td>2</td><td>3</td></tr><tr><td></td><td>3</td><td>2</td><td>1</td></tr><tr><td></td><td>3</td><td>4</td><td>5</td></tr><tr><td></td><td>5</td><td>4</td><td>3</td></tr><tr><td>Unique</td><td>3</td><td>2</td><td>3</td></tr></table>									1	2	3		3	2	1		3	4	5		5	4	3	Unique	3	2	3
																													
	1	2	3																										
	3	2	1																										
	3	4	5																										
	5	4	3																										
Unique	3	2	3																										
MNI = min{3,2,3} = 2																													

In the example, the isomorphisms of the target subgraph or pattern “A-B-A” can be mapped to the example graph as $\{1-2-3\}$, $\{3-2-1\}$, $\{3-4-5\}$ and $\{5-4-3\}$. Thus both nodes labelled A and C can be mapped to three distinct nodes to form a valid isomorphism, namely 1, 3 and 5. However, the second node in the subgraph pattern labelled B can only be mapped to two distinct nodes, numbered 2 and 4. Thus, the MNI of the subgraph pattern is $\min(3, 2, 3) = 2$.

Other support measures such as the Maximum Independent Set (MIS) and Harmful Overlapping (HO) exist in single large graph mining. MIS focuses on the maximum number of non-overlapping matches and HO is based in the number of patterns for which no subgraph is identical. Though all three of these support measures maintain the anti monotonic property, MIS and HO are NP-Hard to compute. On the other hand, calculating MNI of a subgraph is computationally

easier. For this reason, the metric is used in many recent frequent subgraph mining approaches to determine the support of a subgraph in a single large graph [8] [12].

2.3.2 gSpan

gSpan [7] is a frequent subgraph mining algorithm which is designed to work on transactional graph databases. While traditionally, gSpan neither operates on a single large graph setting, nor on weighted graphs, the paper introduced the concepts of edge order and DFS Code and proposed a lexicographical order for subgraphs of a node and edge-labelled graph, based on their *minimum DFS Codes*. This canonical ordering between edges and subgraphs helps avoid duplicate generation of each subgraph, and the DFS Code Tree generated by gSpan's search for frequent subgraph is a concept that can be and has been introduced to the field of Single Large Graphs as well [8]. The canonical ordering of subgraphs introduced in this approach proves to be essential for many subgraph mining algorithms [12]. However, gSpan is not designed for Single Large Graphs and does not take the significance of nodes or edges into consideration.

2.3.3 GraMi

GraMi [8] is one of the most efficient existing frequent subgraph mining algorithms for single large graphs. GraMi uses the MNI value of a subgraph as its support measure, and outputs all subgraphs whose MNI support is equal to or greater than a given user-defined threshold. In GraMi the problem of determining the MNI support of a subgraph is modelled as a CSP (constraint satisfaction problem) which contributes to its efficiency. Furthermore, the authors introduce various pruning

techniques like the push-down pruning, unique labels optimization, automorphism detection, lazy and timed-out searches, and decomposition pruning to make the process of determining if a subgraph is frequent or not less time-consuming.

GraMi operates on edge and node-labelled graphs, and does not take edge-weights into consideration. Thus it fails to identify subgraph patterns containing nodes and edges of high significance.

2.4 Existing Work and their Limitations

In this section, we introduce some existing approaches that focus on mining subgraph patterns from weighted single large graphs, and explore their contributions and limitations.

2.4.1 WIGM

WIGM [11], proposed by *Jiong Yang, Wei Su, Shirong Li and Mehmet M. Dalikic*, works on weighted single large graphs, and identifies subgraphs with high weight values. To define the weight of a subgraph S in a single large graph G , WIGM takes the average of the weights of the edges which are included in at least one valid subgraph isomorphism of S in G .

For an example, let us consider the example large graph in Figure 2.2. For visual ease and simplicity, we suppose that all edges have the same label.

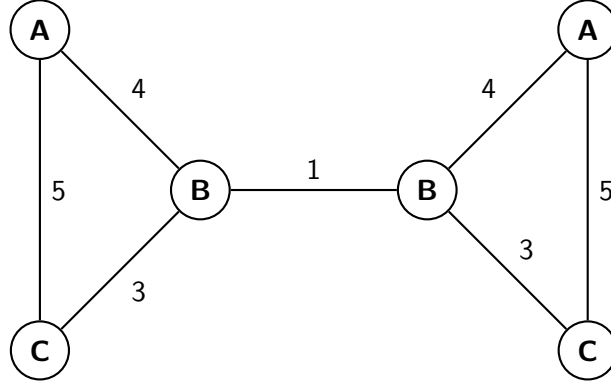


FIGURE 2.2: An Example Graph

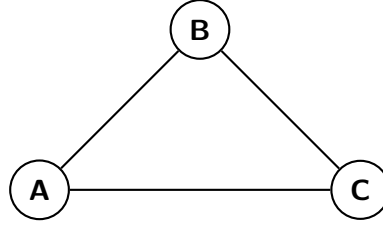


FIGURE 2.3: A Test Subgraph

We can see that there are two subgraph isomorphisms of the subgraph in Figure 2.3 in the example graph of Figure 2.2. The weights of the edges in these two isomorphisms are respectively 5, 4, 3, 5, 4, 3. Thus the weight of the subgraph according to WIGM is $\frac{5 + 4 + 3 + 5 + 4 + 3}{6} = \frac{24}{6} = 4$.

WIGM proposes a bottom up approach to mine subgraphs with high weight values.

Contributions: The authors present a new approach for mining subgraphs with high weight values. That is, it mines subgraphs with weights higher than a user-defined threshold τ , or alternatively, mines the k -highest weighted subgraph patterns. WIGM proves to be one of the earliest works in the field of subgraph mining from weighted single large graphs.

Limitations : WIGM does not take the frequency of the subgraphs mined into consideration. As such, although we know that the subgraphs mined have high weight values, we cannot possibly determine if the subgraph in itself is a regular occurrence or an outlier in the dataset.

2.4.2 ReSuM

ReSuM [12], proposed by *Preti, Lissandrini, Mottin and Velegrakis*, introduces a method to take edge-weights into consideration and mine subgraph patterns which are both relevant and frequent. The working principle of ReSuM is simple, it first uses GraMi bib:grami, an unweighted frequent subgraph mining approach, to find all the frequent subgraphs in a single large graph. It filters these subgraphs and only outputs those subgraphs whose ‘scoring function’ satisfies the user-defined threshold taken as input.

The scoring function f can be any function based on a subgraph pattern P and an input graph G , whose value becomes higher when subgraph isomorphisms of P in G involve edges in G with high weight values.

ReSum requires the scoring function to be *MNI-Compatible*. That is, if the user-given threshold is τ , then $f(P, G) \geq \tau \implies \text{MNI}(P, G) \geq \tau$. Another crucial restriction imposed by ReSuM is that all the edge-weights in the single large graph must be within the interval $[0, 1]$. This implies that the average of the edge-weights of any subgraph pattern shall also fall within the interval $[0, 1]$. Thus, if weighted support is used as the scoring function, and $WS(P)$, $Avg(P)$ and $MNI(P)$ denote the weighted support, average of edge-weights, and the MNI support of a subgraph pattern P respectively, then:

$$\begin{aligned} \text{WS}(P) &= \text{Avg}(P) \times \text{MNI}(P) \\ \implies \text{WS}(P) &\leq \text{MNI}(P) \text{ [Since } \text{Avg}(P) \in [0, 1] \text{]} \end{aligned}$$

This shows that given all edge-weights are in the interval $[0, 1]$, ReSuM can find all subgraphs whose weighted support satisfies a given threshold by blindly running GraMi first and finding all subgraphs whose weighted support is greater than or equal to the threshold.

However, as demonstrated above, restricting all edge-weights to be within $[0, 1]$ means that the weighted support of any subgraph pattern is bounded by its MNI support. This effectively means that we are enforcing the condition that no subgraph that is not frequent itself, can be weighted frequent.

However, this goes against one of the fundamental motivations for taking weights of edges into consideration during frequent subgraph mining. As exemplified in section 1.2.1, many applications require certain subgraphs to be mined with higher priority than others despite their low frequencies due to the presence of edges with high weight values in them. Striking a balance between the frequency of a pattern and its weight is a core concern of weighted pattern mining. Thus, an ideal weighted subgraph mining approach should not only be able to filter out insignificant subgraphs mined by unweighted frequent subgraph mining algorithms, but also mine hidden subgraphs, which although lack in frequency, make up for that deficiency by having high weight values. While ReSuM fulfills the first objectives, it fails to achieve the second.

Strengths:

- Can mine all weighted frequent subgraphs when the edge-weights are within the interval $[0, 1]$.

- Can act as a filtration algorithm that removes insignificant subgraph patterns mined by GraMi.
- Can handle single large graphs where edges have multiple weights assigned to them.

Limitations:

- Cannot mine weighted frequent subgraphs from weighted single large graphs where the edge-weights are not within the range $[0, 1]$.
- Cannot mine subgraphs that are not mined by unweighted frequent subgraph mining approaches due to relatively lower frequencies, but should nevertheless be identified as weighted frequent due to the presence of edges with high weight values in them.
- Offers no counter-measure or effective heuristic to counter the problem of the absence of the *Apriori* property in weighted subgraph mining.
- In order to avoid being faced with the problems created due to the absence of the *Apriori* property, the authors have explicitly defined a family of scoring functions and have specified a continuous range restricting the values that the edge-weights can have $([0, 1])$. These conditions greatly restrict the representational power of weighted single large graphs.

2.4.3 EdWePat

EdWePat[22], proposed by *Bapuji Rao and Sarojananda Mishra*, is an approach where the user can find all isomorphisms of a query pattern in an edge-labelled graph. The authors restrict the query patterns to being chains and simple cycles consisting of exactly three nodes. And despite the title of the paper “An Approach to Detect Patterns (Sub-graphs) with Edge Weight in Graph Using Graph Mining Techniques” suggesting otherwise, the authors practically treat the edge-weights as edge-labels and the quantitative significances of the weight values are ignored.

The approach simply identifies all subgraph isomorphisms of a three-noded query pattern, and does not suggest any pruning steps or optimization measures to speed up the process. The ‘weight’ of each edge in the query pattern must simply match the weight of the edge in the graph it is mapped with during isomorphism.

EdWePat thus bears little to no significance on our work, and is not itself a subgraph mining approach. We acknowledge this paper however due to the surprising recency of its publication, and due to the title of the paper suggesting that is relevant to our work.

Limitations:

- Ignores the quantitative values of edge-weights. Weights are only used here for one-to-one matching, and can be thus thought of effectively being used as edge-labels.
- Does not introduce any pruning or optimization technique.
- Cannot identify isomorphisms of general subgraph patterns. Query patterns cannot have more than three nodes.
- The paper does not contain any results gained from experiments on real-world datasets proving its efficiency, scalability or effectiveness.
- The CSP model introduced in several recent literature offers a larger array of pruning and optimization techniques to solve the problem of identifying if at least k subgraph isomorphisms exist.

Chapter 3

The Proposed Approach

This section contains the detailed description of our proposed method, *WeFreS*. We begin with the introduction to the preliminary concepts related to our work and then proceed to describing the algorithmic details in Section 3.2.

3.1 Preliminary Concepts

In this section we introduce preliminary aspects of our algorithm and define our problem statement and describe our research challenges.

3.1.1 Problem Statement

Given a real valued user defined threshold τ and a weighted single large graph G , we aim to find all frequent subgraphs, S whose weighted support is greater than or equal to τ , that is $WS(S) \geq \tau$.

The concept of a weighted single large graph, G is explained in section 2.3.1.2.

The edge-weights are used as a function of the labels of the nodes connecting the edges in question and represent the significance of the interaction between the entities from the application’s perspective.

For example, if we use the graph in Figure 3.1 with weight functions in Table 3.1 as the input for algorithm with threshold set to 30, the output will be all the subgraphs with weighted support greater or equal to the threshold shown in 3.2

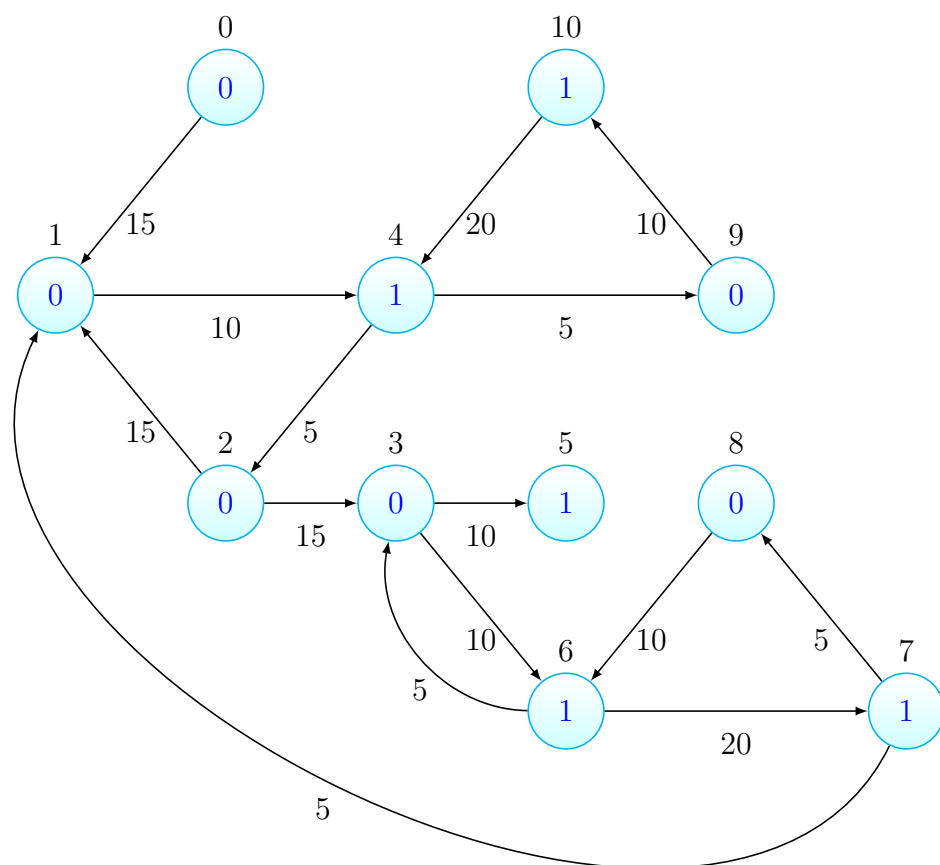


FIGURE 3.1: A Sample Graph

TABLE 3.1: Assignment of Edge-Weights in the Example Graph

Starting Node Label	Ending Node Label	Assigned Weight	Number of Embeddings
0	0	15	3
0	1	10	5
1	0	05	5
1	1	20	2

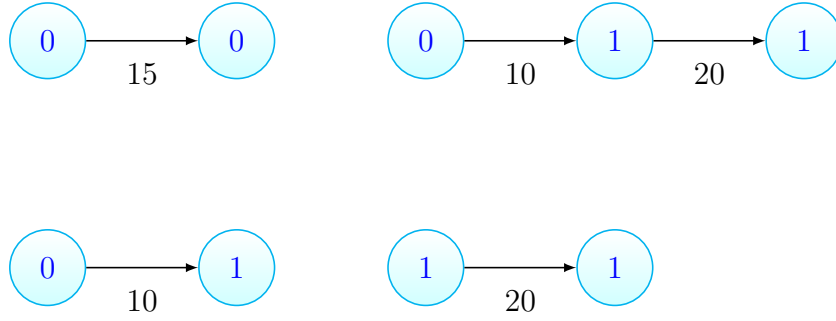


FIGURE 3.2: Output from the Sample Graph with Threshold 30

3.1.2 Research Challenges

Mining frequent patterns from single large graphs is inherently cumbersome. Addition of weights to this process poses an even greater number of research challenges. Two of the main challenges that make the task of applying the concepts used for unweighted single large graph mining to a edge-weighted single large graph model a non-trivial challenge can be listed as follows:

- The *a priori* or downward closure property, which is utilized to reduce the run-time of many frequent pattern mining algorithms including GraMi, is notably absent in a weighted single large graph setting. The downward closure property states that once the support of any itemset reaches below the user defined threshold, all itemsets that may be constructed by adding one or more items to the set will have frequency support less than the threshold. However, this property does not hold true if we are considering the weighted support of a subgraph.

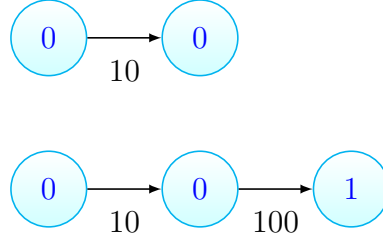


FIGURE 3.3: Absence of Downward Closure Property

The weighted support of a graph is the product of its MNI value and its average weight, that is, $WS(S) = MNI \times Avg(S)$.

While the downward closure property holds for the MNI value, it certainly does not hold for the average weight of a subgraph. In particular, if the current subgraph is extended by an edge with weight far exceeding the current weight of the subgraph, the weight of the resulting extended subgraph will be far greater than the current subgraph, and the extended subgraph may have higher values of weighted support as well.

Considering the two subgraphs in Figure 3.3, we can clearly see the subgraph in the bottom is clearly an extension of the first one. We assume the MNI of the first subgraph is 10, and in accordance with the downward closure property, the MNI support of the second subgraph is 5 which is lower than the subgraph it was extended from.

However, the average weight of the first subgraph is $\frac{10 + 10}{2} = 10$.

The average weight of the second subgraph is $\frac{100 + 10 + 10}{3} = 40$.

Thus, as per our assumptions of the MNI values, the weighted support of the first subgraph is $10 \times 10 = 100$, while the weighted support of the second subgraph, which is its extension is $40 \times 5 = 200$. Thus if the user defined

threshold is 150, the second subgraph will be deemed as weighted frequent even though the first subgraph is not.

As such, the downward closure property clearly does not exist in the context of weighted single large graph mining, and it is not possible to stop extending a subgraph and prune itself and all subgraphs that can be extended from it from the DFS Search tree if a single subgraph is deemed weighted infrequent.

The problem of determining exactly when we can stop recursively extending a certain subgraph, and conclude that no subgraphs extended from it will be weighted frequent is a fundamental concern of **WeFreS**. In this paper, we propose an efficient way to determine the highest possible weight a subgraph can attain after extension by one or more edges from the single large graph, thus making bounded search for the set of weighted frequent subgraphs possible.

- Efficient approaches for mining frequent subgraphs in an unweighted single large graph use the CSP model and instead of finding out the exact MNI of a subgraph, answer the question of whether the MNI value is at least equal to the given user defined threshold. But in the weighted single large graph scenario, the MNI value that is required for a subgraph to be weighted frequent depends on its weight along with the user defined threshold. Also, we now have to answer the following couple of questions for each subgraph in the search space:

- *Is the MNI value of the subgraph sufficient for it to be weighted frequent?*
- *Does the MNI value of the subgraph imply that any subgraph extended by adding one or more edges to it might be weighted frequent?*

So, for every subgraph in **WeFreS** two questions addressing two different MNI values needs to be answered using the CSP model which is designed for a single value constraint.

3.2 Algorithmic Details

3.2.1 Outline of WeFreS

Our proposed algorithm, **Weighted Frequent Subgraph miner, WeFreS** aims to find all weighted frequent subgraphs from a single large graph in an efficient manner.

The basic working principle of **WeFreS** is roughly outlined by Algorithms 1 and 2. Line 1 from Algorithm 1 states that we initially delete all nodes and edges from the input graph from which we are certain that no weighted frequent subgraphs shall ever be produced. The exact process of how we ascertain a node or an edge to be ‘infertile’ is explained in Section 3.2.7.2. In lines 3-5, we initialize a subgraph with each of the remaining edges, and call the recursive function SUBGRAPHMINER.

Algorithm 1 FINDSUBGRAPHS

Input A single weighted large graph G and a user-defined threshold τ .

Output All weighted frequent subgraphs

- 1: Delete all infertile edges and nodes.
 - 2: $result \leftarrow \phi$
 - 3: **for each** $edge \in remaining_edges$ **do**
 - 4: $S \leftarrow edge$
 - 5: $result \leftarrow result \cup SUBGRAPHMINER(S, G, \tau)$
 - 6: **return** $result$
-

Algorithm 2 SUBGRAPHMINER**Input** A test subgraph S , a single weighted large graph G and a threshold τ **Output** All weighted frequent subgraphs that is S or is an extension of S

```

1: if DFSCode( $S$ )  $\neq$  min(DFSCode( $S$ )) then return  $\phi$ 
2:  $result \leftarrow \phi$ 
3: if weighted_support( $S$ )  $\geq \tau$  then
4:    $result \leftarrow S$ 
5: if Extensions of  $S$  may be weighted frequent then
6:    $ext \leftarrow$  All subgraphs formed by adding 1 edge to  $S$ 
7:   for each  $sub \in ext$  do
8:      $result \leftarrow result \cup \text{SUBGRAPHMINER}(sub, G, \tau)$ 
9: return  $result$ 

```

In line 1 of SUBGRAPHMINER, we check if the subgraph S is canonical or not, in order to avoid duplicate subgraph generation. The canonical ordering of subgraphs is a classical concept used in many frequent subgraph mining algorithms.

In lines 3-4, we check if the current subgraph is weighted frequent or not. In line 5, we check if it is indeed possible for a subgraph that is an extension of S to be weighted frequent. If so, we extend the current subgraph and recursively continue our search for weighted frequent subgraphs.

All this being easier said than done, the mining process imposes several challenges. First of all, how do we determine if there is a possibility of an extension of the current subgraph to be weighted frequent. As discussed previously, even though the MNI measure of a subgraph maintains the *Apriori* property, the weight of the extensions of a subgraph may be greater than its own weight. In this work, we introduce a novel approach for determining the **Maximum Possible Weight** (i.e. the *MaxPosW*-measure) a subgraph can attain after being extended by one or more edges.

Once the $MaxPosW$ for a subgraph S is determined, we define $reqExt = \frac{\tau}{MaxPosW}$ as the minimum MNI value S must have for there to be a possibility that a subgraph extended from S shall be weighted frequent. Now, let us suppose that W is the weight of the subgraph S . For S to be weighted frequent itself, we define $reqFreq = \frac{\tau}{W}$ as the minimum required MNI value. Thus, the steps that are required to complete the mining process are the following:

- Defining a method to determine if a node or an edge is *infertile*.
- Defining a method to determine $MaxPosW$ for a subgraph S .
- Defining a method to determine if the MNI support of a subgraph S is greater than or equal to $reqExt$.
- Defining a method to determine if the MNI support of a subgraph S is greater than or equal to $reqFreq$.

Detailed descriptions of all these methods along with necessary mathematical proofs and illustrations are provided in the sections that follow.

3.2.2 Determining an Upper Bound of Maximum Possible Weight

To determine an upper bound, we introduce the $MaxPosW$ -measure, that is the **Maximum Possible Weight** a subgraph can attain after being extended by one or more edges. Our proposed algorithm **WeFreS** uses $MaxPosW$ to reduce search space in finding weighted frequent subgraphs from a single large graph.

3.2.2.1 Calculation of $MaxPosW$

The weight of a subgraph is defined as the average of weights assigned to the edges present in the subgraph. Now, to extend an isomorphic subgraph to be another subgraph isomorphic to the single large graph, only unused edges from the single

large graph are eligible for use during extension.

For example, if we want to extend the subgraph in Figure 3.4 as an isomorphic subgraph to Figure 3.1, only the edges listed in Table 3.2 will be eligible as the rest are already used in the subgraph prior to the extension.

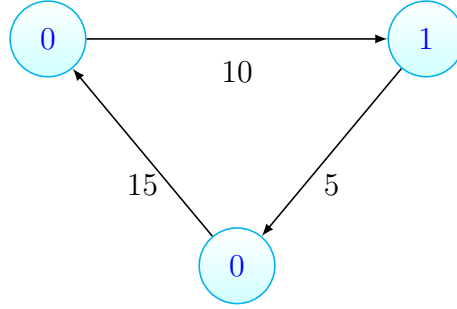


FIGURE 3.4: A Sample Subgraph

TABLE 3.2: Example List of Remaining Edges

Starting Node Label	Ending Node Label	Assigned Weight	Number of Remaining Edges
0	0	15	2
0	1	10	4
1	0	05	4
1	1	20	2

To calculate $MaxPosW$, we need to keep a list edges remaining in the single large graph after subtracting the edges that are already currently in the subgraph. The first challenge in this regard is maintaining a list of the edges remaining. To do so, we consider the recursive structure of the DFS Code Tree, by which we mine all the weighted frequent subgraphs. In a DFS Code Tree, all the children extended from a subgraph are formed by adding a single edge to it. As such, once we recursively call the function and move from a parent to a child subgraph, we are effectively adding a single edge to the subgraph, and deleting an edge from the list of remaining edges. Again, when we backtrack to the parent subgraph, we are

essentially deleting an edge from the child subgraph and adding back an edge to the list of remaining edges in the large graph.

Assuming that we have a data structure capable of fast insertion and deletion to maintain the list of remaining edges (an area that is explored in detail in section 3.2.2.3), our next challenge would be to find the maximum possible weight attainable from the list of edges that are remaining. In this regards, we present Lemma 3.1 :

Lemma 3.1. *The maximum possible weight an existing subgraph can attain after being extended by k -edges is equal to the weight it attains after being extended by the k -edges with the highest edge weights.*

An example use of this lemma 3.1 is presented in Table 3.3.

Initially, as we can see in Table 3.2 when no edges have been removed, the number of edges remaining with weight 20 is 2, with weight 15 is 1, with weight 10 is 4 and with weight 5 is 4. This list of remaining edges can be represented as follows:

2	2	4	4
20	15	10	5

If we extend the current subgraph by exactly one edge, then according to lemma 3.1, we shall select the edge with the highest weight, which is 20. Afterwards, the list of remaining edges would look as follows:

1	2	4	4
20	15	10	5

Lemma 3.1 then leads us to picking the only remaining edge with weight 20 to get the subgraph with highest possible weight after extension by exactly two edges,

TABLE 3.3: Example Use of Lemma 3.1 to Extend the Subgraph in Figure 3.4

Number Edges to Extend k	Edges Used		
	Weight of the Edge	Number of Remaining Edges	Number of Edges to Use
1	20	2	1
	15	2	0
	10	4	0
	5	4	0
2	20	2	2
	15	2	0
	10	4	0
	5	4	0
3	20	2	2
	15	2	1
	10	4	0
	5	4	0
4	20	2	2
	15	2	2
	10	4	0
	5	4	0
...	...		

leaving the list to look as follows:

0	2	4	4
20	15	10	5

With edges of weight 20 no longer there, we must make our third extension with an edge of weight 15.

0	1	4	4
20	15	10	5

The subsequent values not shown in Table 3.3 shall have picked edges with weights 15, 10, 10, 10, 10, 5, 5, 5, 5.

Proof:(By contradiction) Let us assume that the set of k -edges with the highest edge weights are $H_1 = \{h_1, h_2, h_3, \dots, h_k\}$. Let us suppose that the statement is false. Thus, there exists another set of k edges in the large graph $E_1 = \{e_1, e_2, e_3, \dots, e_k\}$, such that $\forall i \exists j (e_j < h_i)$ and $\forall i \forall j (h_i \geq e_j)$ where $1 \leq i, j \leq k$, for which the resulting weight of any subgraph S after all the edges in E_1 are added to it is greater than the resulting weight after all edges in H_1 are added to S .

Let, $sum_h = \sum_{i=1}^k h_i$ and, $sum_e = \sum_{i=1}^k e_i$. It follows that $sum_h > sum_e$. Now, suppose for any subgraph S , with e_{cnt} edges whose sum of weights = e_{sum} , the weight of the subgraph after adding all the edges in H_1 will be $\frac{e_{sum} + sum_h}{e_{cnt} + k}$. Similarly, the weight of the subgraph after adding all the edges in E_1 will be $\frac{e_{sum} + sum_e}{e_{cnt} + k}$. However, since $sum_h > sum_e$, $\frac{e_{sum} + sum_h}{e_{cnt} + k} > \frac{e_{sum} + sum_e}{e_{cnt} + k}$.

Thus, the resulting weight of S after all the edges in E_1 are added to it is less than the resulting weight after all the edges in H_1 are added to it. However, this contradicts with our earlier conjecture. Thus, the statement cannot be false.

Thus the maximum possible average weight a subgraph can attain after being extended by i edges:

$$\mathbf{h}_i = \frac{cur_sum + sum_i}{k + i}$$

where,

- k : the number of edges in the subgraph in question
- cur_sum : the summation of the weights of the k edges present in the subgraph in question
- sum_i : summation of the weights of highest weighted i edges among the remaining edges

Thus, the maximum possible weight attainable by a subgraph after extension by one or more edges,

$$\mathbf{MaxPosW} = \max_{1 \leq i \leq rem}(h_i)$$

where, rem = remaining number of edges

Now, we argue that the distribution of h_i is unimodal. That is, up to a certain point, the value of h_i strictly increases and then, the value of h_i is non-increasing. Alternatively, if for some $0 \leq max \leq rem$, for all $0 \leq i \leq rem$, $h_{max} \geq h_i$, then for all $0 \leq i \leq max$, $h_i \leq h_{i+1}$, and for all $max \leq i \leq rem$, $h_i \geq h_{i+1}$. Theorem 3.2 states this argument.

Theorem 3.2. *Maximum possible average weight a subgraph can attain after being extended by i edges, h_i follows a unimodal distribution.*

The unimodality of h_i in the subgraph in Figure 3.4 can be seen in Table 3.4 and Figure 3.5. Up to the point of $i = 5$ and h_i reaches 17.5, the value of h_i strictly increases and then, the value is always non-increasing.

Proof. We first prove the following statement which is sufficient for implying unimodality. That is, if the following statement is true, the distribution shall be unimodal.

$$\forall i \in 1 \leq i \leq rem, h_i \geq h_{i+1} \Rightarrow h_{i+1} \geq h_{i+2}$$

The proof of sufficiency may be done by induction. Let i be the first value, for which $h_i \geq h_{i+1}$. Thus $\forall 1 \leq j < i, h_j < h_{j+1}$. This indicates that the portion of the distribution before the decreasing part starts, shall be increasing. Now, we prove that, if the aforementioned statement is true, once the non-increasing part appears, the rest of the distribution shall stay non-increasing.

TABLE 3.4: Unimodality of h_i in the Example Subgraph

Number of Edges in the Subgraph, k		3
Summation of the Weights of the k Edges in the Subgraph, cur_sum		30
Number of Edges to Add,	Summation After Extension, sum_i	$h_i = \frac{cur_sum + sum_i}{k + i}$
1	50	12.5
2	70	14
3	85	14.17
4	100	14.29
5	110	17.5
6	120	16.67
7	130	16
8	140	15.45
9	145	14.58
10	150	13.85
11	155	13.21
12	160	12.67

Induction Base:

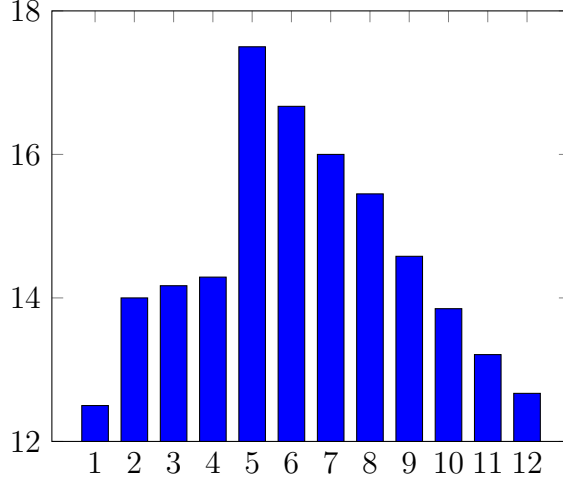
For $k = i + 1$, since $h_i \geq h_{i+1} \Rightarrow h_{i+1} \geq h_{i+2}$ and $h_i \geq h_{i+1}$ is true, thus $h_{i+1} \geq h_{i+2}$ is true[by modus ponens]. This implies that the non-increasing part continues at least upto $i+2$.

Induction Step:

Let us assume, the non-increasing portion continues upto k . That is,

$$h_i \geq h_{i+1} \Rightarrow h_{i+1} \geq h_{i+2} \Rightarrow \dots \Rightarrow h_{k-1} \geq h_k$$

Now by applying modus ponens once again, we can write:

FIGURE 3.5: A Bar Chart Representing the Unimodality of h_i

$$(h_{k-1} \geq h_k \Rightarrow h_k \geq h_{k+1}) \wedge (h_{k-1} \geq h_k) \Rightarrow (h_k \geq h_{k+1})$$

Thus, if the non-increasing part extends upto k , it shall extend upto $k + 1$ as well. By principle of mathematical induction, we can thus conclude that if the statement 3.2.2.1 is proved, once the non-increasing part of the distribution of h_i starts, it stays non-increasing. In simple terms, if statement 3.2.2.1 is true, we can say that upto a certain value of i , the value of h_i increases, and afterwards, it stays completely non-increasing.

Now we prove statement 3.2.2.1, given that $w_i \geq w_{i+1} \geq w_{i+2}$ and h_i is calculated using Equation 3.2.2.1. Here w_k denotes the weight of the k -th edge, when the edges are sorted in decreasing order of edge weights. From Eq. 3.2.2.1,

$$\begin{aligned}
 h_i &= \frac{cur_sum + sum_i}{k + i} \\
 &= \frac{(cur_sum + sum_i)(k + i + 1)}{(k + i)(k + i + 1)} \\
 &= \frac{(cur_sum + sum_i)(k + i) + (cur_sum + sum_i)}{(k + i)(k + i + 1)} \\
 &= \frac{cur_sum + sum_i}{k + i + 1} + \frac{cur_sum + sum_i}{(k + i + 1)(k + i)}
 \end{aligned}$$

$$= \frac{cur_sum + sum_i + \frac{cur_sum + sum_i}{k+i}}{k+i+1} = \frac{cur_sum + sum_i + h_i}{k+i+1}$$

Thus,
$$h_i = \frac{cur_sum + sum_i + h_i}{k+i+1} \dots (i)$$

Eq. (i) also implies that, maximum possible weight after extending the subgraph by considering by (i+1)-edges can at most be,

$$h_{i+1} = \frac{cur_sum + sum_{i+1}}{k+(i+1)} = \frac{cur_sum + sum_i + w_{i+1}}{k+i+1}$$

As, $h_i \geq h_{i+1}$, using Eq. 3.2.2.1 we can write,

Thus, $h_i \geq h_{i+1}$

$$\Rightarrow \frac{cur_sum + sum_i + h_i}{k+i+1} \geq \frac{cur_sum + sum_i + w_{i+1}}{k+i+1}$$

Therefore, $h_i \geq w_{i+1}$

Now,

$$\begin{aligned} h_{i+1} &= \frac{cur_sum + sum_i + w_{i+1}}{k+i+1} \\ &= \frac{\left(\frac{cur_sum + sum_i}{k+i}\right)(k+i) + w_{i+1}}{k+i+1} \\ &= \frac{h_i(k+i) + w_{i+1}}{k+i+1} \geq \frac{w_{i+1}(k+i) + w_{i+1}}{k+i+1} \\ &\geq \frac{w_{i+1}(k+i+1)}{k+i+1} \end{aligned}$$

$$\geq w_{i+1}$$

Therefore, $h_{i+1} \geq w_{i+1}$

Again, from Eq. (i) and using relations $h_{i+1} \geq w_{i+1}$ and $w_{i+1} \geq w_{i+2}$

$$\begin{aligned} h_{i+1} &= \frac{cur_sum + sum_{i+1} + h_{i+1}}{k + (i + 1) + 1} \\ &\geq \frac{cur_sum + sum_{i+1} + w_{i+1}}{k + i + 2} \\ &\geq \frac{cur_sum + sum_{i+1} + w_{i+2}}{k + i + 2} \end{aligned}$$

But, $sum_{i+1} + w_{i+2} = sum_{i+2}$ and so, $\frac{cur_sum + sum_{i+1} + w_{i+2}}{k + i + 2} = \frac{cur_sum + sum_{i+2}}{k + i + 2}$

Using Eq.(i), it follows that, $\frac{cur_sum + sum_{i+2}}{k + i + 2} = h_{i+2}$.

Therefore, $h_{i+1} \geq h_{i+2}$ which completes proving Theorem 1.

Thus distribution of h_i is unimodal.

□

3.2.2.2 Taking Advange of Unimodality

The problem of finding the maximum possible weight a subgraph can attain after being extended by any number of the remaining edges in the main graph can now be reduced to finding the (both global and local) maxima of the unimodal distribution of h_i . But running a linear search for the maxima will be inefficient for large graphs with a large number of edges.

As such, we take advantage of the unimodality, and propose applying the classical approach of ternary search on the range $[1, \text{rem}]$, to find the maximum possible weight that can be attained by extending the current subgraph by one or more edges from the main single large graph. Here, ternary search works in $O(\log_2(\text{rem}))$.

The pseudo code for ternary search is provided in Algorithm 3.

Algorithm 3 TERNARYSEARCH

Input A function $h(x)$ and the number of remaining edges in the graph rem

Output Max value of $h(x)$ for $1 \leq x \leq \text{rem}$

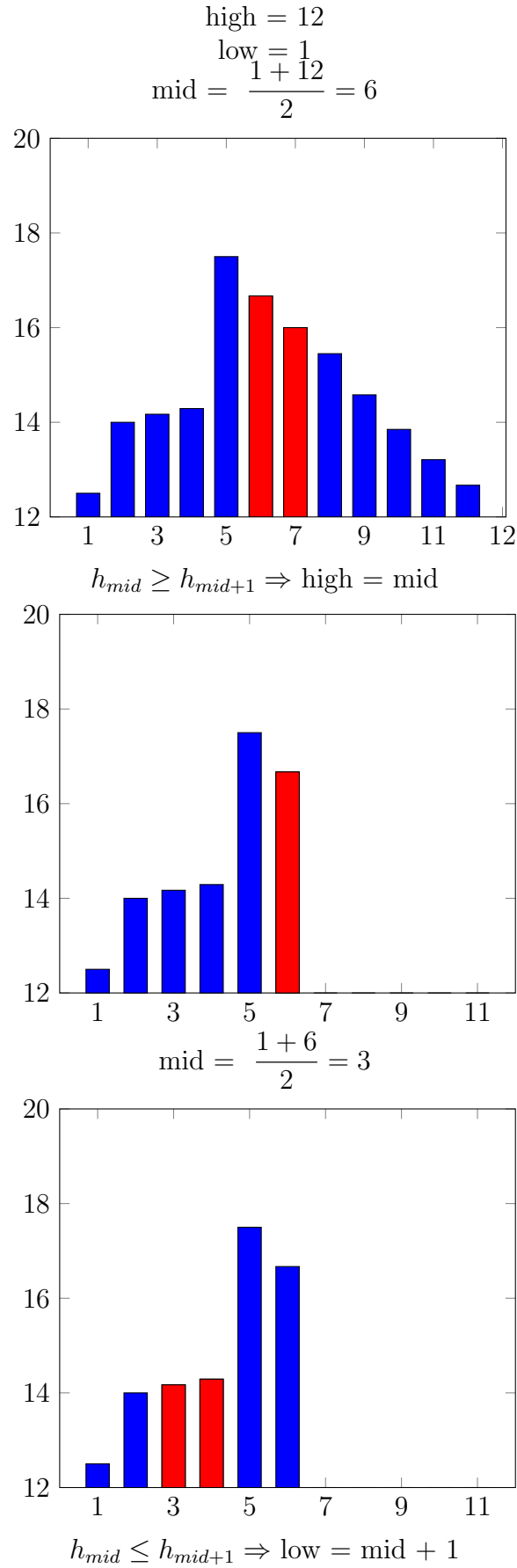
```

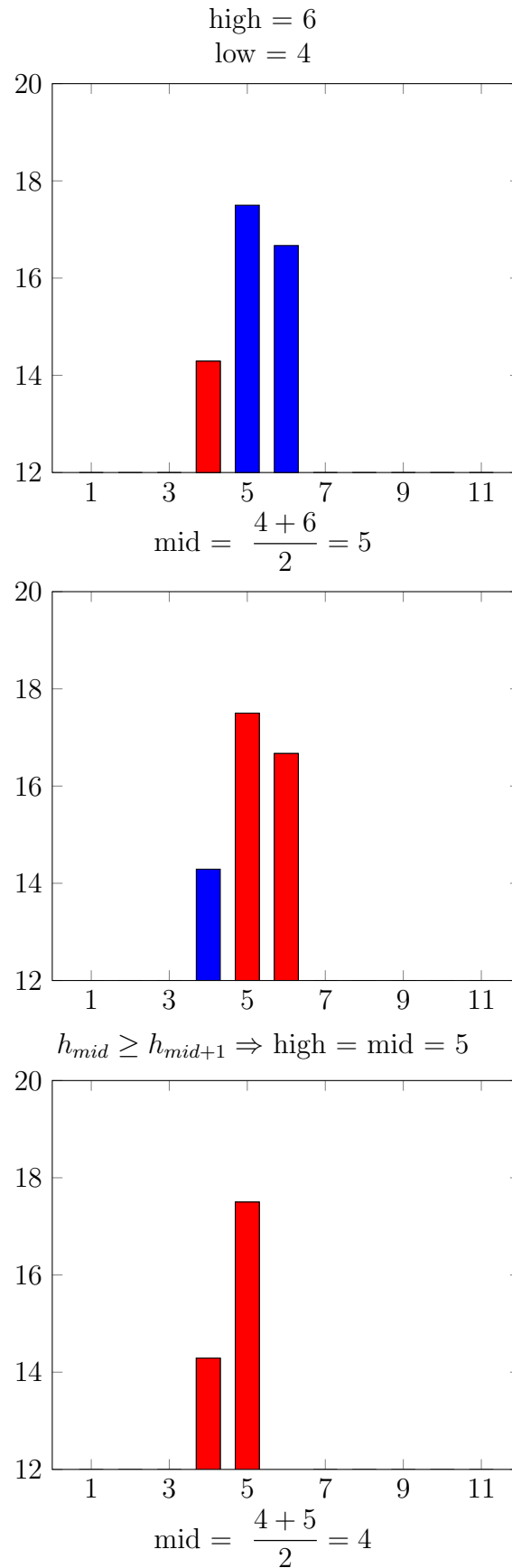
1: high  $\leftarrow$  rem
2: low  $\leftarrow$  1
3: while high > low do
4:   mid  $\leftarrow$  (high+low)/2
5:   if ((mid=rem) || (h(mid)  $\geq$  h(mid+1))) then
6:     high  $\leftarrow$  mid
7:   else
8:     low  $\leftarrow$  mid + 1
9: return high

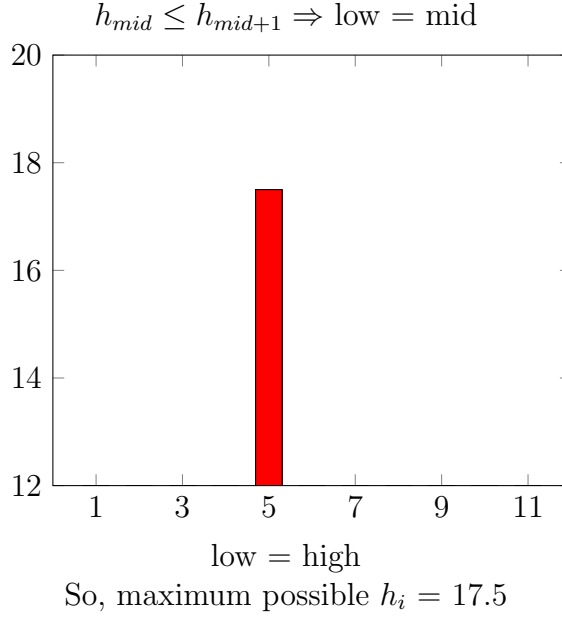
```

Ternary searching for the maxima in this case is akin to searching the lowest value of i where $h_i \geq h_{i+1}$

An demonstration of ternary search according to algorithm 3 on h_i from Table 3.4 is shown sequentially in figure 3.6, 3.7 and 3.8.

FIGURE 3.6: Demonstration of Ternary Search on h_i (i)

FIGURE 3.7: Demonstration of Ternary Search on h_i (ii)

FIGURE 3.8: Demonstration of Ternary Search on h_i (iii)

3.2.2.3 Maintaining a Sorted List of Remaining Edges

As discussed earlier, during the algorithm's inorder traversal through a DFS Code Tree, once we add an edge to the current subgraph, and proceed to a child subgraph (in order to check if the child is frequent or not, or to check if any subgraph formed by adding edges to the child subgraph may be frequent), we are effectively removing an edge from the sorted list of edges remaining in the main subgraph. Once we backtrack from the child subgraph to the parent subgraph, we are effectively adding the edge back to the list of remaining edges. Again, while calculating h_i , during the ternary search, we require the value of sum_i , that is the sum of i highest weighted edges in the remaining list of edges.

Thus we need a data structure capable of supporting fast insertion, deletion and efficiently answering a query regarding the sum of the first i values in a list.

For these actions, we use a segment tree or an interval tree. To reduce the size of the segment tree, we group edges with equal weights together, and keep count of

how many edges map to each distinct weight. For example, the graph in Figure 3.1 has edges with 4 distinct weight values, which are $\{20, 15, 10, 5\}$.

Each node of the segment tree has two children except the leaf and shall contain the sum of the edges weights in the interval it represents, and the number of active edges present in the interval. Let n be the initial number of distinct weights.

The initialization of the segment tree built for the graph in Figure 3.1 when there is no subgraph in question is demonstrated in Figure 3.9.

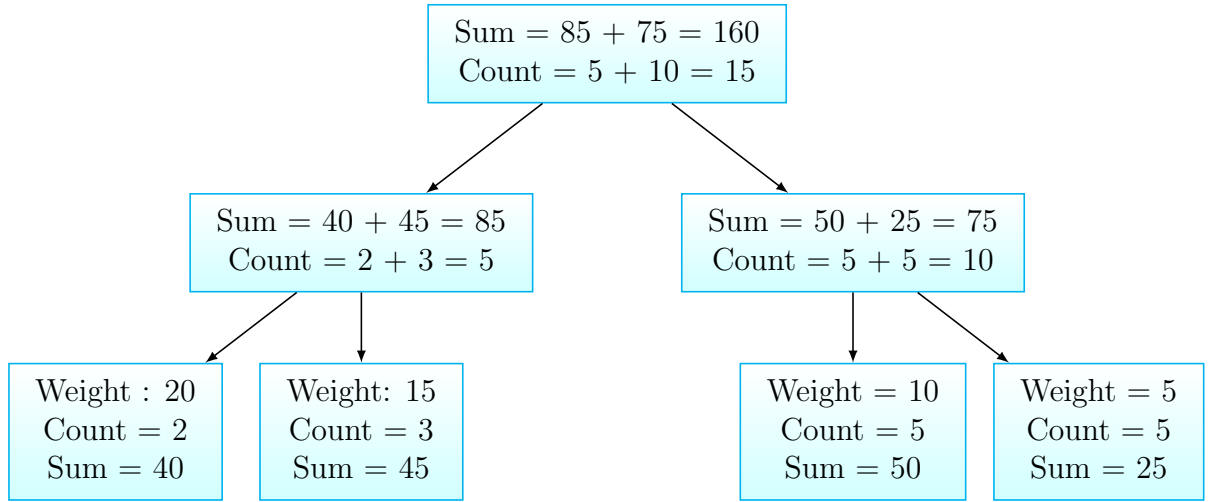


FIGURE 3.9: Initialization of the Segment Tree for the Graph in Figure 3.1

So, before subgraph extension, we initially have the segment tree in presented in Figure 3.10 for the example graph in Figure 3.1.

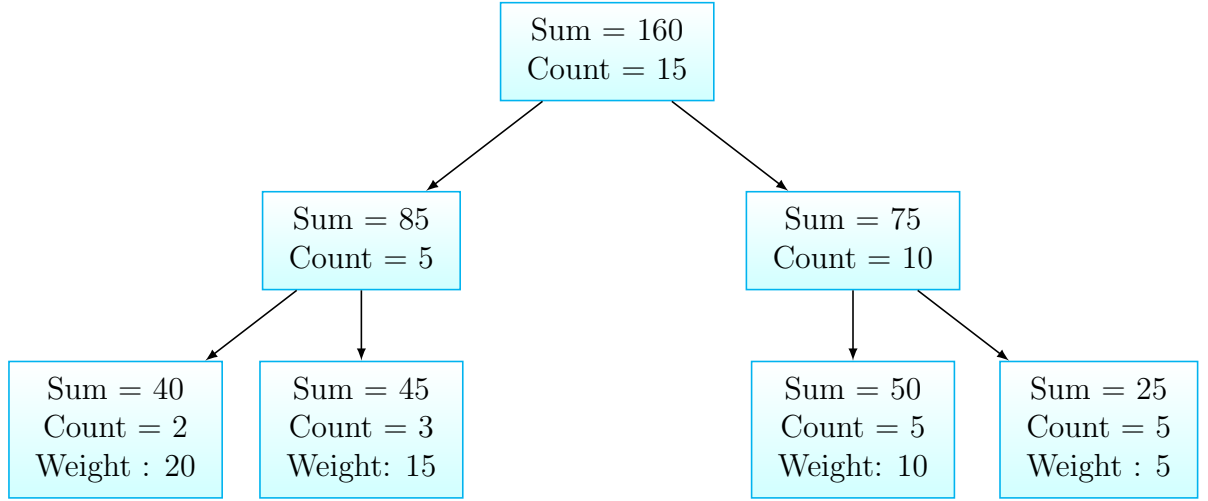


FIGURE 3.10: The Initial Segment Tree for the Graph in Figure 3.1

Addition of an edge to a subgraph is equivalent to deleting the edge from the list of remaining edges. This is done by decrementing the count value from the leaf node representing the same weight value as the edge used for extension and changing the ancestors accordingly

For example, if we use an edge with weight 15 like Figure 3.11 from the graph in Figure 3.1 to create the subgraph in Figure 3.11, the segment tree in Figure 3.10 changes to the segment tree in Figure 3.12.

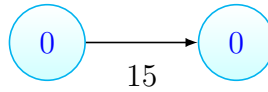


FIGURE 3.11: An Initial Edge Added to Create a Subgraph

An insertion operation is equivalent to the classical $O(\log_2 n)$ segment tree operation of increasing the count and sum of all the nodes of the segment tree which contain the weight of the edge in its interval.

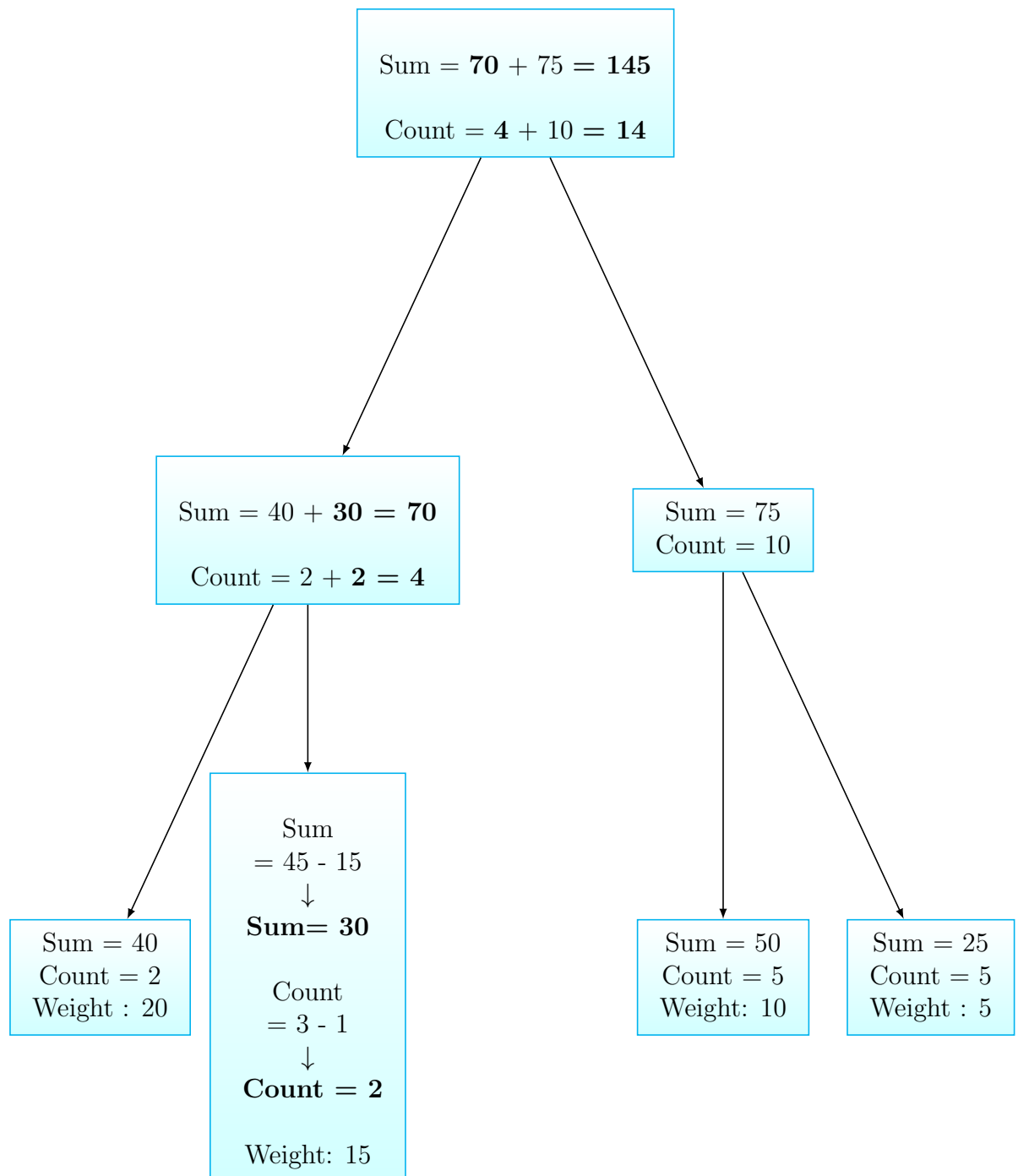


FIGURE 3.12: Deletion: The Segment Tree for the Graph in Figure 3.1 after the Creation of the Subgraph in Figure 3.11.

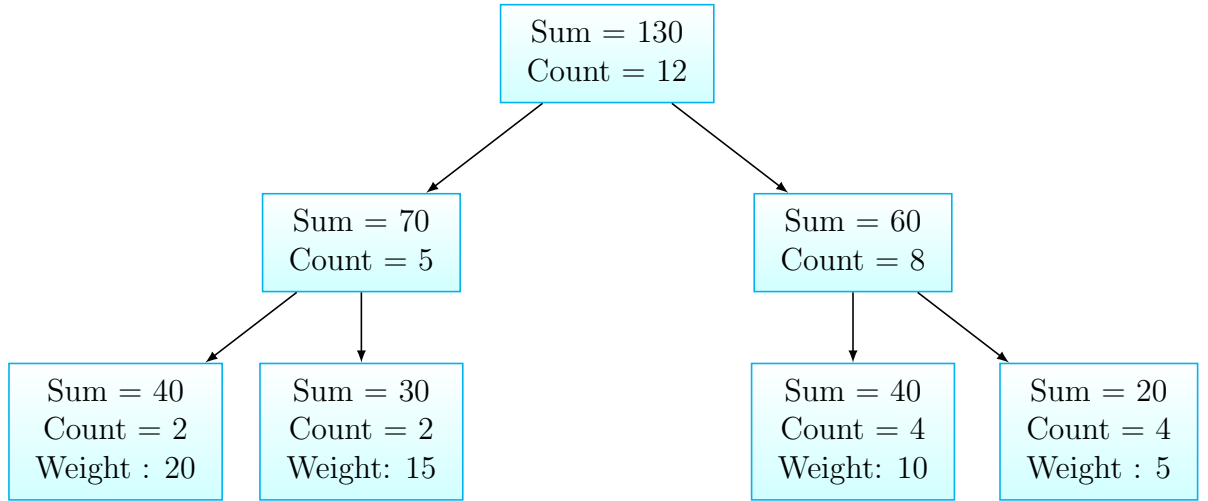


FIGURE 3.13: The Segment Tree for the Graph 3.1 after Subgraph 3.4

Now, for constructing the subgraph in Figure 3.4, three edges are used for extension and removed from the list of remaining edges one by one in similar manner. The current state of the segment tree with the subgraph in Figure 3.4 is demonstrated by Figure 3.13.

When a subgraph backtracks to its parent subgraph, the edge previously used for extension is again available for creating a different subgraph. So, the edge must be added to the list of remaining edges. The edge is added to the list by incrementing the count value from the leaf node representing the same weight value as the edge used for extension and changing the ancestors accordingly

For example, if we remove the edge in Figure 3.11 from the subgraph in Figure 3.4 to backtrack to the subgraph in Figure 3.14, the segment tree in Figure 3.13 changes as shown in Figure 3.15.

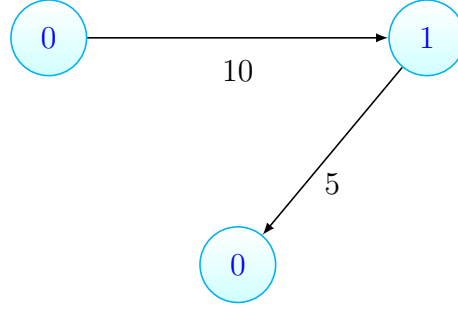


FIGURE 3.14: Subgraph Constructed after Removing the Edge in Figure 3.11 from the Subgraph in Figure 3.4

A deletion operation is equivalent to the classical $O(\log_2 n)$ segment tree operation of decreasing the count and sum of all the nodes of the segment tree which contain the weight of the edge in its interval.

After the deletion operation, the segment tree reaches the state shown in Figure 3.16.

So, the use of segment tree ensures efficient insertion and deletion from the list of remaining edges. The remaining challenge in this regard, is to figure out the sum of the first i active edges in the segment tree in an efficient manner.

To determine this sum, we require an $O(\log_2 n)$ traversal of the segment tree from the root to the leaf the i -th active edge is in. The recursive query function described below defines how each query is processed by the segment tree. Initially we call `QUERY(root of segment tree, i)` shown in algorithm 4, to get the sum of the first i edges.

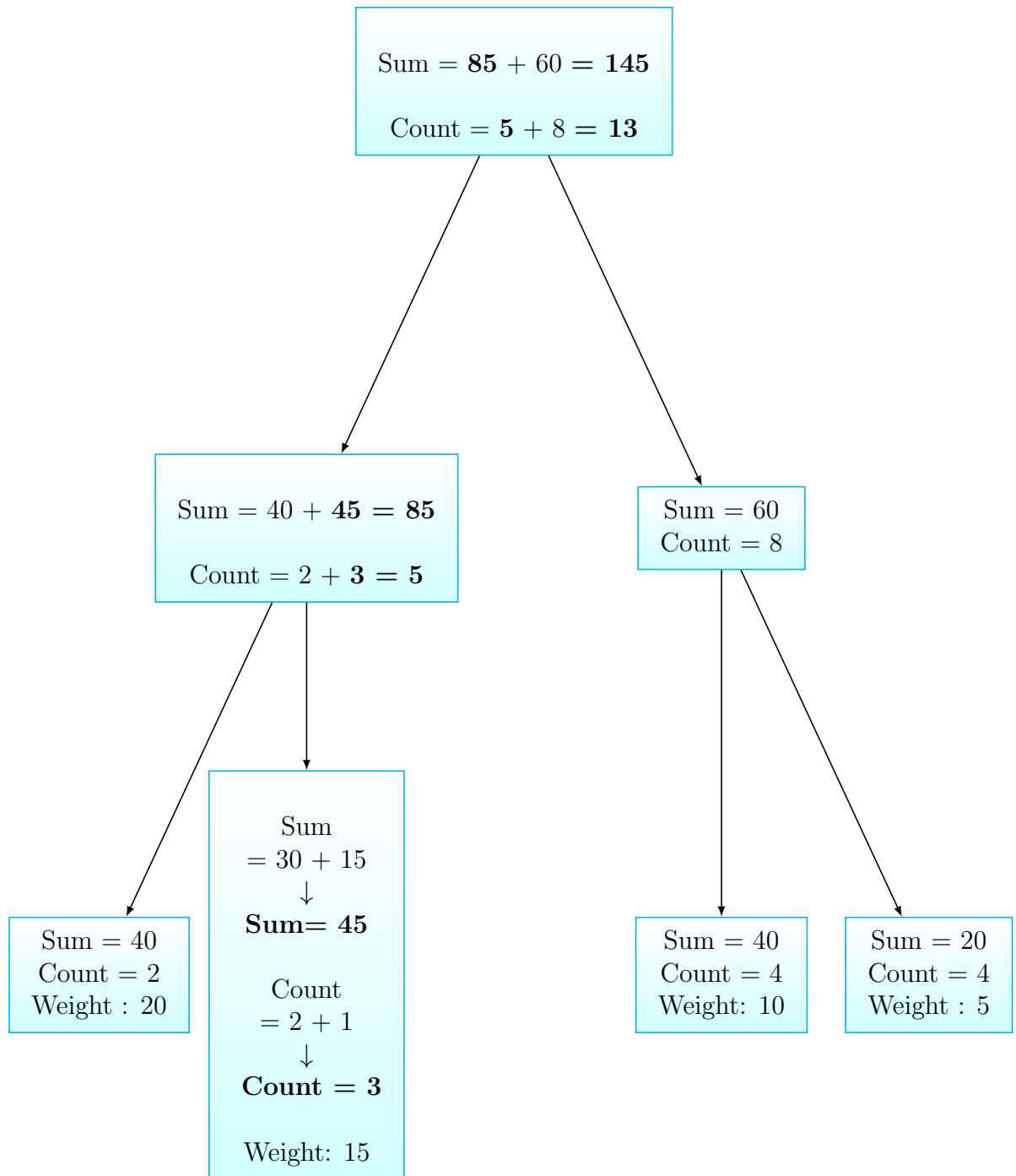


FIGURE 3.15: Insertion: The Segment Tree after Removing the Edge in Figure 3.11 from the Subgraph in Figure 3.4.

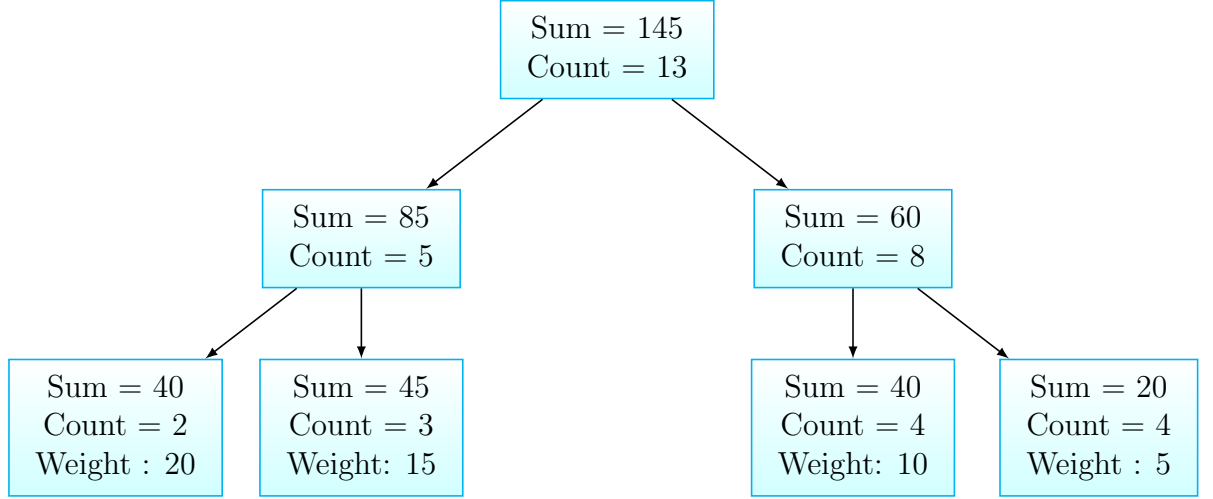


FIGURE 3.16: The Segment Tree for the Graph in Figure 3.1 after Creating the Subgraph in Figure 3.4

Algorithm 4 QUERY

Input Current node of the segment tree $curr$, number of element in subtree to be searched val

Output Sum of first val nodes in subtree of $curr$

```

1: if  $curr$  is a leaf then
2:   return sum of  $val$  edges
3:  $leftNode \leftarrow$  left child of  $curr$ 
4:  $rightNode \leftarrow$  right child of  $curr$ 
5: if  $leftNode.activeEdges \geq val$  then
6:   return QUERY( $leftNode, val$ )
7: return  $leftNode.sum + \text{QUERY}(rightNode, val - leftNode.activeEdges)$ 
  
```

The recursive function follows three basic principles:

1. If the current node is a leaf, output the sum of val edges here. Edges sharing the same leaf have the same weight. (Lines 1-2)
2. If left child has enough active edges, move to the left child. (Lines 5-6)
3. Else take the sum of all left child's active edges, move to right child and adjust val accordingly. (Line 7)

For example, if we want sum_i where $i = val = 6$ for the subgraph in Figure 3.14, the query will be done on the segment tree in Figure 3.16.

We will start from the root. In the segment tree, the number of active edges is represented by *Count*. $leftNode.activeEdges = 5$ and $val = 6$. So, take the sum of all left child's active edges, move to right child and adjust val accordingly. Now, $val = val - leftNode.activeEdges = 6 - 5 = 1$.

For the current node, $leftNode.activeEdges = 4$ and $val = 1$. As the left child has enough active edges, we move to the left child.

The current node is a leaf. So, with $val = 1$ we return the sum of the first edge, which is 10.

So, returning to the previous node, sum is returned as $= leftNode.sum + QUERY(rightNode, val - leftNode.activeEdges) = 85 + 10 = 95$.

3.2.3 Determining a Lower Bound of the MNI of a Subgraph

3.2.3.1 Identifying a Weighted Frequent Subgraph

As per our earlier discussion, a subgraph is considered frequent if the product of the current weight and frequency or MNI of the subgraph is greater than or equal to a given threshold. Here,

$$WS(S) = sW(S) \times sG(S)$$

$$\text{or, } WS(S) = Avg(S) \times MNI(S)$$

As the threshold, τ is known and the average weight of a subgraph can be easily calculated, we need to know the MNI of the subgraph to make the decision. Now, for a subgraph to be frequent,

$$WS(S) \geq \tau$$

$$\text{or, } Avg(S) \times MNI(S) \geq \tau$$

So, there exists a minimum value of MNI for a subgraph to be frequent. If we denote this value as required MNI or frequency to be considered frequent, $reqFreq$, we can write

$$currentWeightofSubgraph \times reqFreq = \tau$$

$$\text{or, } reqFreq = \lceil \frac{\tau}{currentWeightofSubgraph} \rceil$$

So, we do not need the exact MNI of a subgraph to check if it is frequent or not. Rather, we only need to check if there are at least $reqFreq$ instances of the subgraph in question.

For example, the average weight of edges for the subgraph in Figure 3.4, $currentWeightofSubgraph = \frac{15 + 10 + 5}{3} = 10$. If the threshold, τ is set to 30, required MNI for this subgraph to be frequent, $reqFreq = \lceil \frac{30}{10} \rceil = 3$.

So this subgraph will be considered frequent if there is at least 3 instances of it in the single large graph in Figure 3.1.

3.2.3.2 Identifying the Extendability of a Subgraph

As, we have discussed earlier, minimum support index or MNI of a subgraph maintains the downward closure property, but weighted support does not. So, a frequent subgraph can be an extension of an infrequent subgraph.

Having an efficient way to maintain the list of remaining edges, we are able to quickly find the maximum possible weight that a subgraph can attain after being extended by some number of these edges.

Now, for the current subgraph, no extension of the subgraph can achieve more average weight than the maximum possible weight, $MaxPosW$ using the remaining edges. As the MNI of a subgraph is anti-monotonic, a subgraph extended

from another subgraph will have the same number of instances or less in the single large graph as the parent subgraph and the maximum possible MNI is equal to the current MNI. So, it is possible to extend a frequent subgraph from the current subgraph if the following property is true.

$$MaxPosW \times MNI \geq \tau$$

In other other, for an extension of a subgraph to be frequent, the current subgraph needs to have a MNI value greater than or equal to a required value which can be denoted as *reqExt*. So, it is possible to extend a frequent subgraph from the current subgraph if

$$MaxPosW \times reqExt = \tau$$

$$\text{or, } reqExt = \lceil \frac{\tau}{MaxPosW} \rceil$$

For example, from Table 3.4, we can see that the maximum possible weight for the subgraph in Figure 3.4, $MaxPosW = 17.5$. If the threshold, τ is set to 30, required MNI for this subgraph to be frequent, $reqExt = \lceil \frac{30}{17.5} \rceil = 2$.

So this subgraph will be considered for extension if there is at least 2 instances of it in the single large graph in Figure 3.1.

Again, there is no need to find the exact MNI of a subgraph to check if it is extendable or not. Rather, we only need to check if there are at least *reqExt* instances of the subgraph in question.

3.2.4 The CSP Model

We have defined two MNI lower bounds, *reqFreq* - the required MNI for being frequent and *reqExt* - the required MNI for being extendable. In both cases, checking if at least same number of instances of the subgraph as the MNI exists in the single

large graph is sufficient.

We check if the MNI is a certain value at minimum by modeling the problem as a constraint satisfaction problem or CSP. The subgraph in question is marked as the constraint graph and each solution of the CSP denotes a subgraph isomorphism to the single large graph.

But we need to consider two MNI values instead of one. So, we could run the CSP model twice and make decisions based on the separate results. But using the CSP model twice means taking double amount of time compared to finding subgraph isomorphisms in unweighted single large graphs.

Suppose, we have two values: the *maxMNI* which represents the maximum of the two values (required MNI values for being frequent and extendable) and the *minMNI* which represents the minimum of the same.

When the problem is modelled as a constraint satisfaction problem, we list each node of the subgraph as a variable, and the nodes in the large graph that have the same labels as them form their respective domains. A valid assignment mapping each element in the subgraph to a node from its domain must satisfy the following constraints:

1. No two different nodes in the subgraph can be mapped to the same node in the large graph.
2. The label of each node in the subgraph must match the label of the node in the large graph it is matched to.
3. For each edge (s_1, s_2) in the subgraph, if v_1 is the large graph node assigned to s_1 and v_2 is assigned to s_2 , there must exist an edge (v_1, v_2) in the large graph.

For example, if the subgraph in Figure 3.14 is in question, in the CSP model

we consider this graph as the constraint graph. So, the variables are label 0 and 1.

The domains for label 0 and 1 are the sets of nodes $\{0,1,2,3,8,9\}$ and $\{4,5,6,9\}$ respectively.

The valid mappings for this subgraph to the graph in Figure 3.1 are given in the Table 3.5.

0	1	0
1	4	2
1	4	9
3	6	7
8	6	3

TABLE 3.5: Valid Mappings of the Subgraph in Figure 3.14

For the subgraph in Figure 3.4 , $reqFreq = 3$ and $reqExt = 2$.

So, for this subgraph, $maxMNI = 3$ and $minMNI = 2$.

After defining $maxMNI$ and $minMNI$, we confine two CSP models into one using a status flag.

3.2.4.1 Using a Status Flag

To calculate the MNI, we observe the number of nodes that each variable can be assigned to, such that at least one valid subgraph isomorphism can be formed from the assignment.

Initially, we seek the answer to the question of whether the MNI is the subgraph is at least equal to the $maxMNI$. To enumerate this search, we set the *statusFlag* to 2. We iterate over each variable, and check if at least one valid assignment can be formed by assigning at least $maxMNI$ distinct elements from its domain.

Once such *maxMNI* values are found for a single variable, we move on to the next variable immediately.

If all the values in the domain of a node have been exhaustively searched, and *maxMNI* valid assignments have still not been found, we change the *statusFlag* to 1, meaning that we are now searching for satisfaction of *minMNI*. For every variable onwards, we search for *minMNI* distinct values that can be assigned to them to form valid assignments. If for any variable, such *minMNI* elements are not found, the *statusFlag* is set to 0, and the search for figuring out a lower bound of the MNI value is immediately terminated, with the return value indicating neither MNI values were satisfied.

The process is summarized in Figure 3.17.

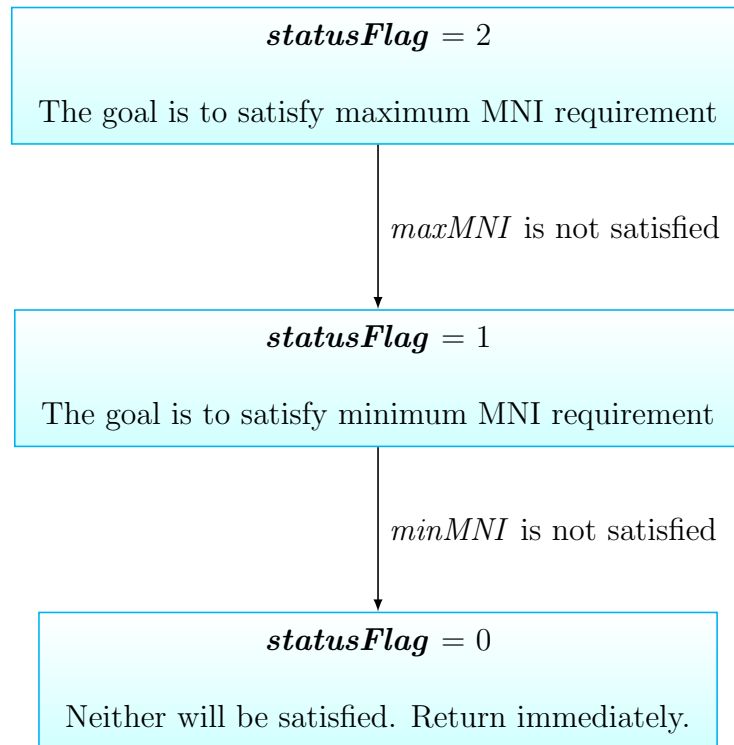


FIGURE 3.17: Using a Status Flag with the CSP Model

The following algorithm titled MNILOWERBOUND in Algorithm 5 represents the detailed working procedure of the CSP model according to our defined MNI lower bounds. Our model preserves the optimizations of push-down pruning [line 2], unique labels optimization [line 3], automorphism detection [line 5] and optimization [lines 9-11], lazy and timed out searches [line 18-20] and decomposition pruning [line 31] used in recent frequent subgraph mining algorithms that are based on single large graphs[8][12], and keeps track of the MNI value to be satisfied using the *statusFlag* [lines 6, 13, 29-30, 43-44].

The status flag, *statusFlag* is initialized as 2 [line 6]. If it is there are less members in the domain with the minimum size than the required *maxMNI* or $\max\tau$, status flag is changed to 1 as it is not possible to satisfy *maxMNI* [line 7-8]. While checking each element in the domain, if the current or possible domain size indicate that the current MNI in question can not be satisfied, the *statusFlag* is changed according to the flow of Figure 3.17 [line 16,33,45]. If at any moment the *statusFlag* is set to 0, the procedure returns 0 and terminates immediately.

For example, for the subgraph in Figure 3.4, *maxMNI* = 3 and *minMNI* = 2. According to the Table 3.5 when checking the domain of label 1, *maxMNI* is not satisfied and the *statusFlag* will be changed to 1 from 2. This will save the time when compared to individually searching for MNI = 3 and MNI = 2. In the end, 2 will be returned from the function. Thus, we are able to draw conclusions about the subgraph calling CSP only once using decision conditions discussed later.

Algorithm 5 MNILOWERBOUND

Input A subgraph S , a single large graph G , a minimum frequency threshold $min\tau$, and a maximum frequency threshold $max\tau$

Output A lower bound of $MNI(S)$

```

1: Consider the subgraph  $S$  to graph  $G$  CSP and apply node and arc consistency
2: Apply push down pruning
3: if  $S$  and  $G$  satisfy the unique labels optim. conditions then
4:   return minimum size among all domains
5: Compute the automorphisms of  $S$ 
6:  $statusFlag \leftarrow 2$  //Looking to satisfy  $max\tau$ 
7: if minimum size among all domains  $< max\tau$  then
8:    $statusFlag \leftarrow 1$  //Looking to satisfy  $min\tau$ 
9: if minimum size among all domains  $< min\tau$  then return 0
10: for each variable  $x$  and its domain  $D$  do
11:    $count \leftarrow 0$ ,  $timedoutSearch \leftarrow \emptyset$ 
12:   if there is an automorphism with a computed domain  $D'$  then
13:      $D \leftarrow D'$ 
14:     move to the next variable
15:   Apply arc consistency
16:   Change  $statusFlag$  according to the domain size
17:   for each element  $u$  of  $D$  do
18:     if  $u$  is already marked then
19:        $count++$ 
20:     else
21:       Search for a solution that assigns  $u$  to  $x$  for a given time threshold
22:       if search timeouts then
23:         save the search state in a structure  $timedoutSearch$ 
24:       if a solution  $Sol$  is found then
25:         mark all values of  $Sol$  to the corresponding domains
26:          $count++$ 
27:       else
28:         remove  $u$  from the domain  $D$ 
29:         add  $u$  to the invalid assignments of  $D$  in  $S$ 
30:       if  $count$  matches the searched value then
31:         Move to the next variable
32:   if  $|timedoutSearch| + count \leq$  the searched value then
33:     change  $statusFlag$  accordingly

```

```

34:   Apply decomposition pruning
35:   for each  $t \in \text{timeoutSearch}$  do
36:       Resume search from the saved state  $t$ 
37:       if a solution  $Sol$  is found then
38:           Mark all values of  $Sol$  to the corresponding domains
39:            $count++$ 
40:       else
41:           Remove  $u$  from the domain  $D$  and add  $u$  to the invalid assignments
           of  $D$  in  $S$ 
42:       if  $count$  matches the searched value then
43:           Move to the next variable
44:   if  $count <$  the searched value then
45:       change  $statusFlag$  accordingly
46: return  $count$ 

```

3.2.5 Decision Conditions

From the above discussion, we can conclude that for a subgraph S , the minimum required MNI for being frequent, $reqFreq$ and the minimum required MNI for extension, $reqExt$ can be determined in the following way:

$$reqFreq = \lceil \frac{\tau}{currentWeightofSubgraph} \rceil$$

$$reqExt = \lceil \frac{\tau}{MaxPosW} \rceil$$

We declare a subgraph to be weighted frequent if and only if the MNI of the subgraph is at least equal to $reqFreq$.

We extend a subgraph if and only if the MNI of the subgraph is at least equal to $reqExt$.

Instead of using two CSP models, we use a single call to the procedure for two different MNI values. So, from the result from MNILOWERBOUND in algorithm 5, two questions need to be answered:

a) *Is the current subgraph frequent?*

b) Is it possible to extend the current subgraph using the remaining edges to construct a frequent subgraph?

Now, the following algorithm 6 called GETMNISTATUS indicates the answers to these questions in numeric representation.

Algorithm 6 GETMNISTATUS

Input A subgraph S , a single large graph G , required frequency for being frequent $reqFreq$, required frequency for being extendable $reqExt$

- 1: $mniCount \leftarrow \text{MNILOWERBOUND}(s, G, \min(reqFreq, reqExt), \max(reqFreq, reqExt))$
 - 2: **return** 0 if $MNI(S)$ is below $reqFreq$ and $reqExt$
 - 3: **return** 1 if $MNI(S)$ is below $reqFreq$ and above $reqExt$
 - 4: **return** 2 if $MNI(S)$ is below $reqFreq$ and above $reqExt$
 - 5: **return** 3 if $MNI(S)$ is above $reqFreq$ and above $reqExt$
-

For any subgraph, these four return values denote four different possibilities.

- 0 represents the subgraph is neither frequent nor extendable.
- 1 represents the subgraph is not frequent, but can be extended to a frequent subgraph.
- 2 represents the subgraph is frequent, but there are no frequent extensions of it.
- 3 represents the subgraph is both frequent and extendable.

For example, for the subgraph in Figure 3.4, function 5 named MNILOWERBOUND returns 2.

We have previously seen that, $reqFreq = 3$ and $reqExt = 2$

$$reqFreq \geq MNI(S) \text{ and } reqExt \leq MNI(s)$$

So, GETMNISTATUS according to algorithm 6 will return 1 which denoted the subgraph is not frequent, but can be extended to a frequent subgraph.

3.2.6 Deletion of Infertile Node and Edge Labels

Our algorithm, **WeFreS** defines a node label or a label pair as infertile when it is mathematically impossible for any subgraph with that node label or the label pair to be a part of a weighted frequent subgraph. To avoid unnecessary calculations, such infertile node labels and labels pairs are deleted from the node label and label pair list before starting the extension process.

As per our earlier discussion, for any extension from the current subgraph to be frequent, the product of $MaxPosW$ and MNI of the current subgraph must be greater than or equal to the weighted support threshold, that is, $MaxPosW \times MNI \geq \tau$. This is also true for a subgraph containing only one edge. So, label pairs are marked infertile if the product of the total number of the said label pair in the large graph and $MaxPosW$ is lower than the given threshold, T and all edges with such label pairs are deleted from the list.

Node labels that are not present in fertile or extendable label pair can not contribute to any frequent subgraphs. These labels are marked as infertile and nodes with an infertile label is removed to avoid unnecessary calculation

The entire process is represented by the pseudo code in 7.

Algorithm 7 DELETEINFERTILELABELS**Input** Set of edges $Edges$, list of labels $labelList$ **Output** List of edges and labels which are not infertile

```

1: for each  $e \in Edges$  do
2:   Initialize subgraph  $S$  with edge  $e$ 
3:   Decrement count of edge  $e$  in  $SegmentTree$ 
4:   Get both node labels as  $labelA$  and  $labelB$  and edge-label  $labelE$ 
5:   Get  $domainSize$  for  $labelA$ ,  $labelB$  and  $labelE$ 
6:   if  $\min(domainSizeA, domainSizeB, domainSizeE) \times MaxPosW(S) \leq \tau$ 
       then
7:     remove  $e$  from  $Edges$  and Segment Tree
8:   else
9:     Increment count of  $e$  in  $SegmentTree$ 
10: for each  $label \in labelList$  do
11:   if label has no adjacent edges in  $Edges$  then
12:     remove  $label$  from  $labelList$ 
13: return  $Edges$  and  $labelList$ 

```

3.2.7 Complete Design of WeFreS

3.2.7.1 Initiating the Algorithm

WeFreS takes a weighted graph G and the minimum support threshold, τ as input. To begin the process of weighted frequent subgraph mining, the steps in algorithm 8 called FREQUENTWEIGHTEDSUBGRAPHMINING are followed.

While unweighted pattern mining procedures sort node labels by their frequency, we propose sorting the labels by the sum of the weights of the edges adjacent to each label, since such labels are more likely to output a greater number of weighted frequent subgraphs. Also, edge relations containing these labels are thus removed earlier, in line 15, thus reducing the number of edges with high weight values earlier, and this in turn helps reduce the maximum possible weight estimation for subsequent subgraphs. The sorting and relabelling of vertices is done in lines (1-3).

Algorithm 8 FREQUENTWEIGHTEDSUBGRAPHMINING**Input** Weighted graph G and the minimum weighted support Threshold τ **Output** All subgraphs S of G such that $W(S) \cdot MNI(S) \geq \tau$

-
- 1: Find *adjacentWeightSum* of each node label
 - 2: *labelList* $\leftarrow$ Set of all node labels in G , sorted in decreasing order of *adjacentWeightSum*
 - 3: Relabel the vertices according to index in *labelList*
 - 4: *result* $\leftarrow \emptyset$
 - 5: Let *Edges* be the set of all edges of G
 - 6: *sortedEdgeList* $\leftarrow$ *Edges* sorted in decreasing order of weight
 - 7: Compress *sortedEdgeList*
 - 8: Build *SegmentTree* with *sortedEdgeList*
 - 9: Sort *Edges* in DFS Lexicographical Order
 - 10: *Edges*, *labelList* $\leftarrow$ DELETEINFERTILELABELS(*Edges*, *labelList*)
 - 11: **for each** $e \in$ *Edges* **do**
 - 12: Initialize subgraph s with edge e
 - 13: Decrement count of edge e in *SegmentTree*
 - 14: *result* $\leftarrow$ *result* $\cup$ SUBGRAPHEXTENSION(e , G , τ)
 - 15: remove e from *Edges* and Segment Tree
 - 16: **return** *result*
-

Lines 5-8 are concerned with sorting the edges according to their weights, compressing edges with equal weights, and building the segment tree, as mentioned in section 3.2.2.3

Line 9 sorts the edges according to the DFS Lexicographic Order.

Line 10 deletes infertile labels from the label list, and edges connecting pairs of labels from which no weighted frequent subgraph can be derived.

In lines 11-15, we iterate over each edge. Line 12 initializes a subgraph with a single edge, line 13 deletes that edge from the segment tree (decrements its count, and adjust the weights accordingly). Line 14 calls the recursive function SUBGRAPHEXTENSION which returns a list of weighted frequent subgraphs derived after extending the current edge, and line 15 deletes that edge from the segment tree (sets its count to 0, and adjusts the weight accordingly).

3.2.7.2 Subgraph Extension

In section 3.2.7.1, we introduced the recursive function SUBGRAPHEXTENSION which returns a list of weighted frequent subgraphs derived after extending the current edge. The details of this function are given in algorithm 9.

Algorithm 9 SUBGRAPHEXTENSION

Output All subgraphs that are S or are extended from S , with weighted support exceeding $\geq \tau$

```

1: if DFSCode( $S$ )  $\neq$  min(DFSCode( $S$ )) then
2:   return
3:  $reqFreq \leftarrow \lceil \frac{\tau}{currentweightofsubgraph} \rceil$ 
4:  $reqExt \leftarrow \lceil \frac{\tau}{maxPosW(S)} \rceil$ 
5:  $mniLowerBound \leftarrow MNILOWERBOUND(S)$ 
6:  $result \leftarrow \emptyset$ 
7: if  $mniLowerBound \geq reqFreq$  then
8:    $result \leftarrow S$ 
9: if  $mniLowerBound \geq reqExt$  then
10:  for each edge  $e$  in  $Edges$  and node  $u$  of  $S$  do
11:    if  $e$  can be used to extend  $u$  then
12:      Let  $ext$  be the extension of  $S$  with  $e$ 
13:      Decrement the count of  $e$  in  $SegmentTree$ 
14:       $result \leftarrow result \cup SUBGRAPHEXTENSION(ext, G, \tau)$ 
15:      Increment the count of  $e$  in  $SegmentTree$ 
16: return  $result$ 

```

In line 1 of the SUBGRAPHEXTENSION procedure, we check if the DFS code of the current subgraph is lexicographic minimum or not. The concept of DFS code and their lexicographical order was defined by gSpan. We terminate the procedure if the DFS code is not lexicographic minimum (line 2). Lines 3 and 4 determine the minimum weight a subgraph must have to be weighted frequent and extendable. By extendable, we mean that a subgraph extended from it could be weighted frequent, and that such subgraphs need to be searched into. The minimum required MNI value for being extendable is derived from the calculation

of the *MaxPosW*, as described in section 3.2.2.1.

We calculate a lower bound of the MNI value of the given subgraph as described in section 3.2.3 in line 5. We add it to the list of weighted frequent subgraphs if this lower bound satisfies the requirement for being so (line 7-8).

In case the lower bound satisfies the required MNI value for being extendable, we recursively extend the current subgraph by generating all possible subgraphs that can be formed by adding a single edge to it. Before each recursive call, we decrement the count of the edge being used to extend the subgraph from the segment tree and adjust the weights accordingly. After the recursive call completes, we increment the count of that edge, and adjust the weights again. These are done to keep the list of remaining edges consistent, as described in section 3.2.2.3. The procedure returns the list of subgraphs which are weighted frequent and are either the current subgraph itself or can be derived by extending it.

3.3 Time Complexity of WeFreS

The time complexity of **WeFreS** depends on the number of subgraphs visited by Algorithm 9 multiplied by the time required for frequency evaluation of each subgraph pattern by Algorithm 5. Here, we denote the number of nodes in the input graph and the subgraph pattern as V_G and V_S respectively and the number of edges in the input graph as E_G . The number of subgraph patterns generated by the DFS Code Tree in the worst case is bounded by $O(2^{V_G^2})$. **MaxPosW** is determined in $O((\log_2(E_G))^2)$ time as described in Section 3.2.2, which is negligible in terms of the overall complexity. As for the time complexity of determining the lower bound by Algorithm 5, applying the push-down pruning, automorphism checking and unique labels optimizations are done in $O(V_S^2)$, $O(V_S^{V_S})$ and $O(V_S)$ time, as is done in GraMi. Since $V_S \ll V_G$, the effect of these added complexities is negligible. Supposing that the probability of the MNI of a subgraph satisfying the maximum MNI requirement $\max\tau$ is p_1 , if the probability of finding a valid isomorphism by mapping a node in the subgraph to a value in its domain is p , then

for each node in the subgraph, the expected number of domain values for which we must search for an isomorphism is $p_1 \cdot \left(\frac{\max \tau}{p}\right) + (1 - p_1) \cdot \frac{\min \tau}{p}$. The complexity of the search for isomorphism is $O(V_G^{V_S-1})$, thus the overall complexity for Algorithm 5 is $O(V_S \cdot (p_1 \cdot \left(\frac{\max \tau}{p}\right) + (1 - p_1) \cdot \frac{\min \tau}{p}) \cdot V_G^{V_S-1})$. If τ_1 is the maximum of all $\max \tau$ found in each subgraph, and τ_2 is the minimum of all $\min \tau$, the overall complexity of **WeFreS** is bounded by $O(2^{V_G^2} \times V_S \cdot (p_1 \cdot \left(\frac{\tau_1}{p}\right) + (1 - p_1) \cdot \frac{\tau_2}{p}) \cdot V_G^{V_S-1})$.

Chapter 4

Experimental Results and Analysis

This section contains the detailed analysis and results of experiments we conducted on real-world graph datasets to evaluate the performance of our proposed approach **WeFreS** in comparison with other existing approaches and baseline algorithms. Runtime, memory consumption and search space are used as metrics for comparison. The search space of an algorithm is simply the number of subgraphs for whom it was necessary to determine the lower bound of the MNI support.

4.1 Data Description

The experiments were conducted on three different real-world datasets that may be represented as single large graphs. A brief discussion of the nature of the datasets and what they represent is given below:

- **MiCo** represents a co-authorship and collaboration graph crawled by the authors of GraMi [8] from *academic.research.microsoft.com*. It is an undirected graph where each node represents an author, an edge represents a

collaboration between two authors in an academic research, and the label of each node represents the author’s field of interest.

- The **Amazon** [39] co-purchase network consists of items found in the *Amazon* website. Each node represents an item, and an edge between two nodes indicate that the two items were either purchased together or one after the other by the same user. We consider here the subgraph for electronic products, and the dataset can be found in *jmcauley.ucsd.edu/data/amazon/*.
- The **FreeAssoc** [40] dataset consists of a network between words used in the English language. It was formed by a survey that asked over 6000 participants to name the word that first comes up to their mind which was strongly associated or meaningfully related to a certain given word. For example, a participant on coming across a given word ‘fire’, may react with the word ‘burn’. The words are represented by nodes in the graph and participant interactions help connect one word with another, which is represented by an edge. The dataset was collected from *Pajek datasets*.

Quantitative information regarding the datasets are shown in Table 4.1. As seen below, all three datasets represent single large graphs, and their differing features help us evaluate the performance and robustness of **WeFreS** in a number of different real-world applications. All edges were considered to have the same labels.

TABLE 4.1: Dataset Specifications

Dataset	No. of Nodes	No. of Edges	No. of Nodelabels	Directed
Mico	100,000	1080,298	29	No
Amazon	164,305	297,112	1856	Yes
FreeAssoc	10,618	63,612	11	Yes

4.1.1 Weight Inclusion

Since none of the three datasets had pre-specified edge-weights, we generated the edge weights ourselves. We assigned negative exponentially distributed edge-weights to the *Mico* dataset and normally distributed edge-weights on the *Free-Assoc* dataset. Both weight distributions are applied to the *Amazon* dataset, and the weights in the Amazon dataset are normalized.

Using different strategies to assign edge-weights was done with the view of evaluating the performance of **WeFreS** in a variety of different weight ranges. The fact that in real-life situations, users are generally much more interested in learning about co-authorship information among authors in certain academic fields while many other collaborations in other academic fields remain of little interest, was used as the intuition behind assigning weights to the *MiCo* dataset using a negative exponential distribution. Having a negative exponential distribution ensures that only a few types of collaborations have high weights, while most others are expected to have relatively low weights.

The *FreeAssoc* dataset, modelling word associations in the English language was assigned weights according to a normal distribution. Thus, most pairs of associated words were assigned relatively medium weights, with very few word pairs being assigned extremely high or extremely low weights.

The *Amazon* dataset was experimented on using both statistical weight distributions, and then normalizing the weights. In addition to ensuring the performance evaluation of **WeFreS** on low weight values, the normalization helps us compare the performance of **WeFreS** to the existing approach *ReSuM*, which requires all edge-weights to be in the range $[0,1]$. Table 4.2 contains the specifications regarding the weight values assigned to each dataset.

TABLE 4.2: Weight Distributions and Specifications

Dataset	Weight Distribution	μ	σ	λ	MinW	MaxW	Normalized
Mico	Neg. Exp.	-	-	1.0	25.75	70.0	No
Amazon	Normal	10	1	-	0.0	1.0	Yes
	Neg. Exp.	-	-	1.0	0.0	1.0	
FreeAssoc	Normal	25	1	-	22.99	27.49	No

Finally, to show that **WeFreS** can perform sufficiently well even in an unweighted single large graph, and that the measures we introduce here introduce little additional overhead to the existing approach for mining frequent subgraphs from unweighted single large graphs, we compare **WeFreS** to *GraMi* by assigning weight equal to 1 in all edges of all three datasets.

4.2 Environmental Setup

Experiments on the Amazon and FreeAssoc datasets were conducted on a device with 8GB RAM, an intel core i5 7th gen processor, with clock speed 2500 MHz clock speed and an Ubuntu 17.10 operating system. Experimental results on the unweighted version of all three datasets were performed on this device as well.

Experiments on the Mico dataset after introducing edge-weights were conducted in a device with 64GB RAM, intel core i7-7820X processor with 3600 MHz clock speed running on 8-cores and an Ubuntu 16.04 LTS operating system.

4.3 Performance Evaluation

The performance of **WeFreS** was evaluated using runtime, memory consumption and search space reduction as the primary comparison metrics. The performance evaluation is subdivided into three parts. Section 4.3.1.1 compares **WeFreS** with two baseline algorithms which can mine weighted frequent subgraphs from single large graphs having any real-valued weights on edges. Section 4.3.1.2 uses the

normalized weight values in the *Amazon* dataset to compare it with the existing approach *ReSuM* which only allows MNI-compatible scoring functions and imposes the constraint that weights must have values within the closed interval $[0, 1]$. Section 4.3.1.2 compares our proposed method with GraMi in an unweighted single large graph, and shows that although *WeFreS* is originally intended for use in a weighted single large graph, its performance remains comparable to a state-of-the-art unweighted frequent subgraph mining approach even in the absence of weights.

4.3.1 Comparison with Weighted Frequent Subgraph Mining Approaches

4.3.1.1 Comparison with Baseline Algorithms

To the best of our knowledge, as of now there is no existing weighted frequent subgraph mining approach that works on single large graphs having edge-weights of any numeric value. As such, we have defined two baseline algorithms for comparison, namely the *MaxW-FSM* approach and the *MaxW-DoubleCSP* approach.

Following the same principle as other weighted pattern mining approaches such as WFIM, both these algorithms use *maxW*, the highest weight among all edge weights in the large graph instead of using our proposed *maxPosW* measure. In most cases, the *maxW* measure is an extravagant over-estimation which results in increased search space, runtime and memory consumption. The benefits of using the *maxPosW* measure instead are observed in the experimental results that follow.

The difference between the two approaches is in the way they calculate the lower bound of the MNI support of each subgraph. *MaxW-DoubleCSP* uses the CSP model used in unweighted approaches and issues two calls to determine if the subgraph's MNI support is enough to make it weighted frequent and if the MNI support is enough to make it extendable. *MaxW-FSM* uses the CSP model we

define in section 3.2.3, and thus issues a single CSP call only.

Comparisons of the runtimes of the baseline algorithms with **WeFreS** is shown in

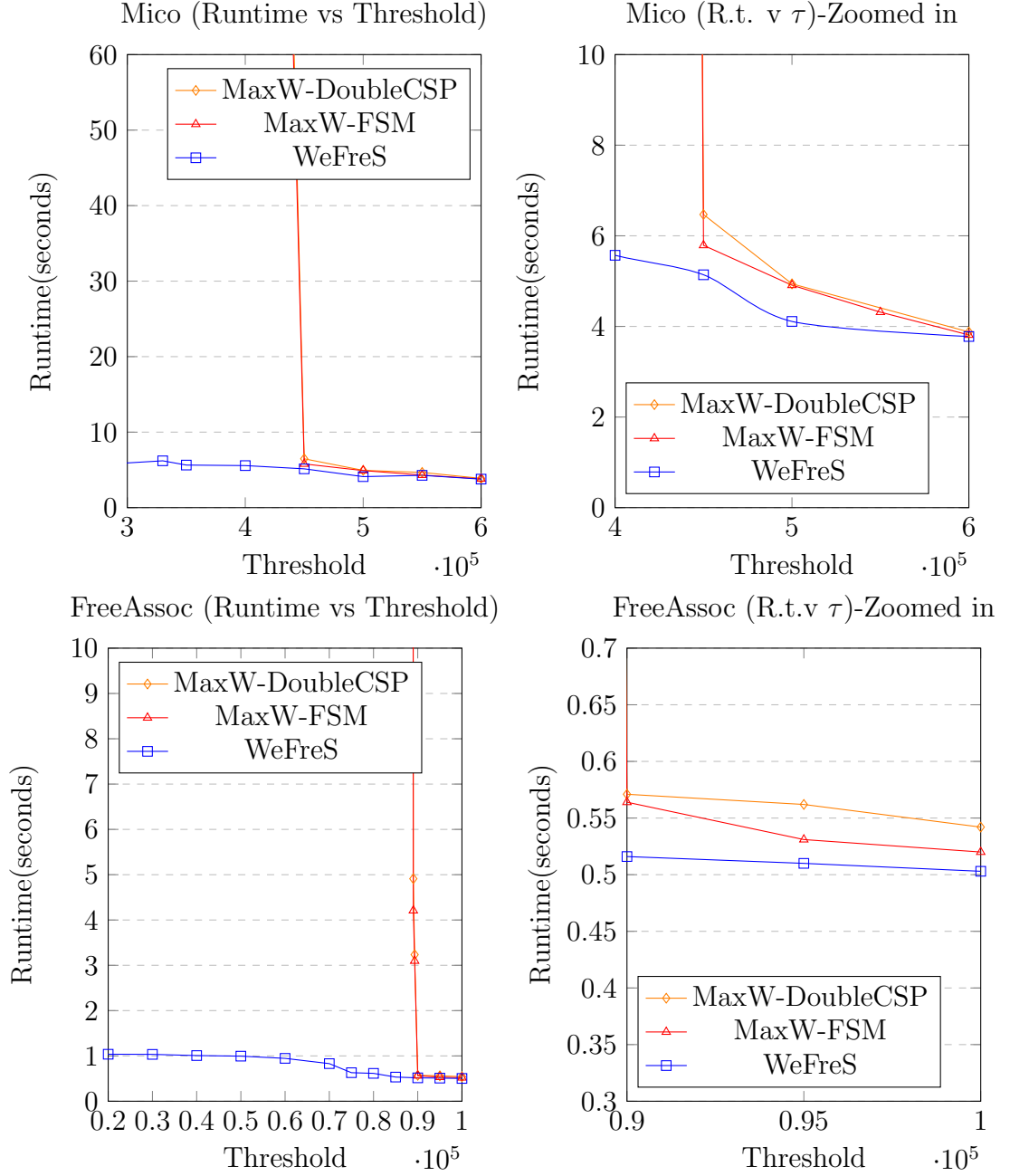


FIGURE 4.1: Runtime Comparisons with Baseline Algorithms

Figure 4.1. From the figures, it is clear that **WeFreS** significantly outperforms the

baseline algorithms in terms of runtime. The sharp spike in the runtimes of both *MaxW-DoubleCSP* and *MaxW-FSM* represents the fact that, when the threshold becomes relatively low neither baseline algorithm could produce output even after being run for hours. **WeFreS** on the other hand, produced sound and complete results within a matter of seconds.

Figure 4.2 demonstrates the number of subgraphs each algorithm can mine when limited by a time constraint. The results once again show that WeFreS can mine many more weighted frequent subgraphs than the baseline algorithms, which struggle to mine any weighted frequent subgraph at all from datasets as large as MiCo and FreeAssoc.

The major improvement in runtime happens due to the reduction of search space achieved by using the *MaxPosW* measure. Figure 4.3 compares the search space of the algorithms. The search space of MaxW-FSM and MaxW-DoubleCSP remains the same since they both use the *maxW* measure. As seen in the figure, even at relatively high thresholds the search space of the MaxW algorithms rises sharply, thus making them both slower and more memory consuming.

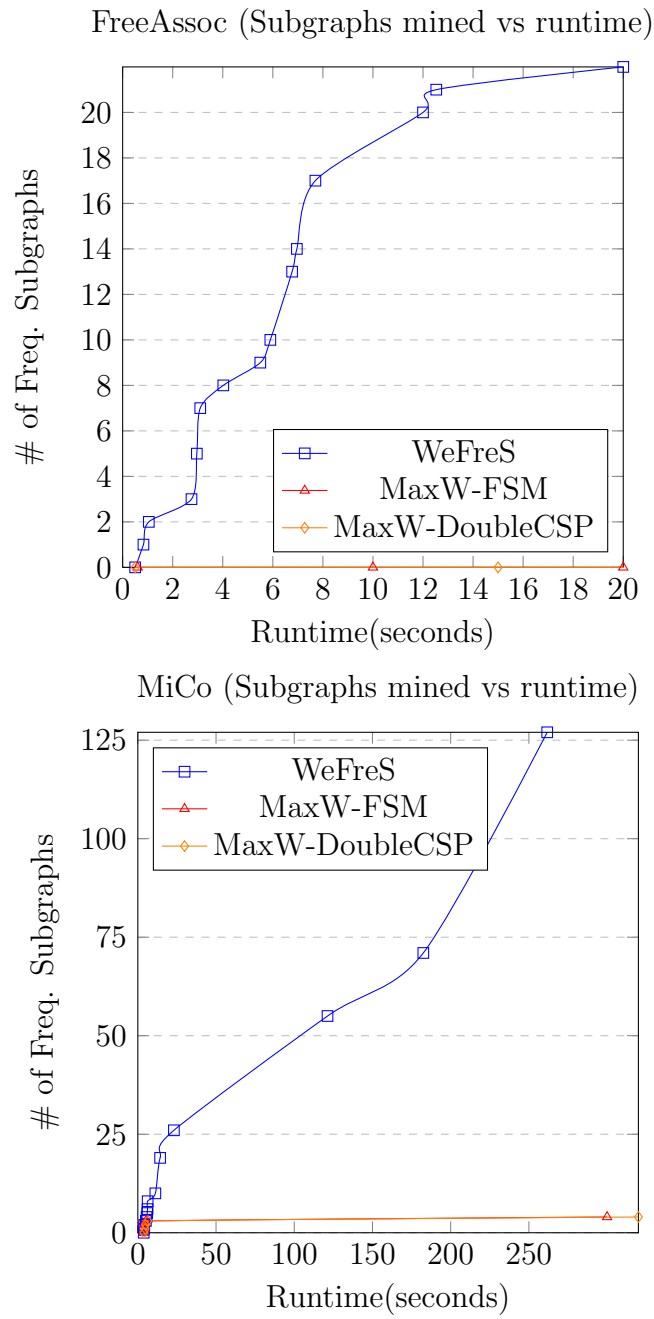


FIGURE 4.2: Number of Frequent Subgraphs Mined in a Given Time Constraint

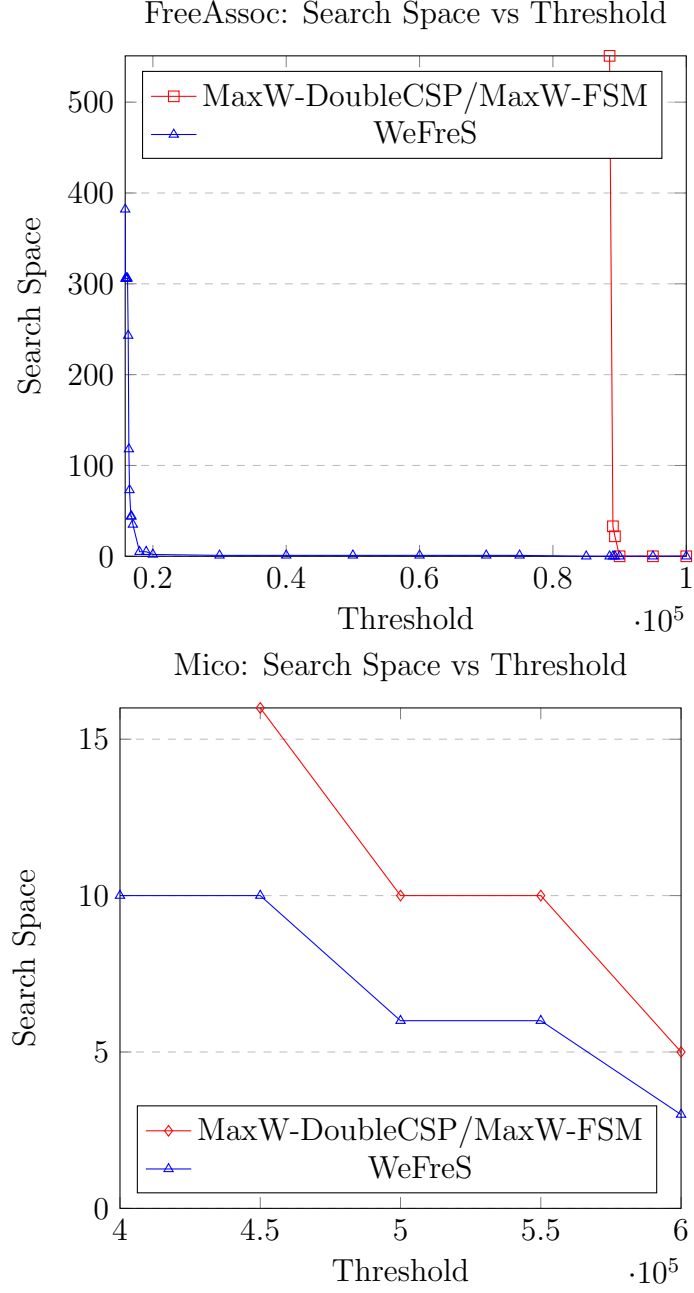


FIGURE 4.3: Search Space Comparison with Baseline Algorithms

In general, increased search space leads to increased memory consumption in frequent subgraph mining algorithms due to optimization measures such as the push down pruning technique being in place. The comparison of the memory consumption of WeFreS with MaxW-FSM is given in Table 4.3. The cells marked ‘X’

TABLE 4.3: Memory Usage Comparisons

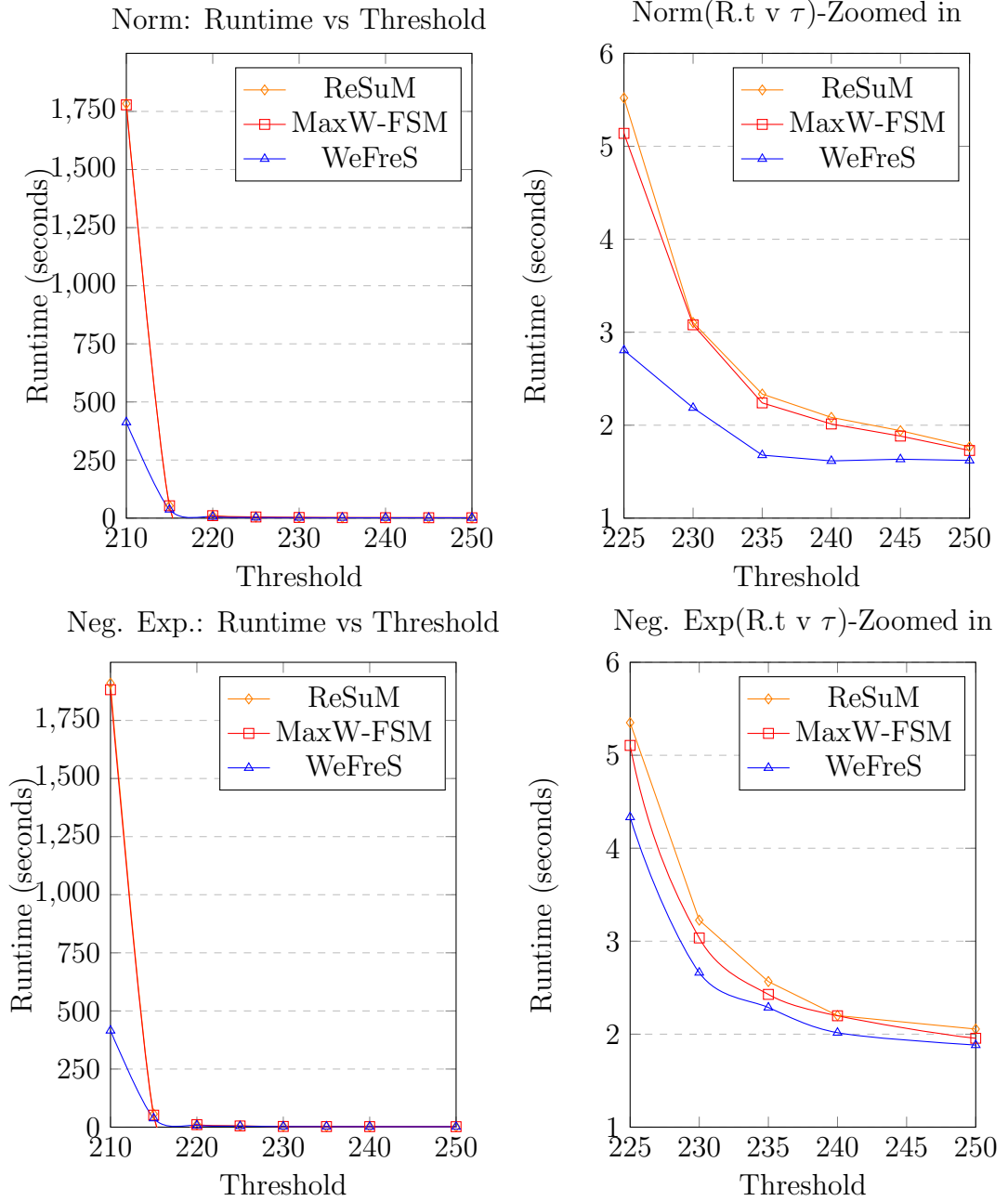
Dataset	Threshold	Memory Usage(WeFreS) (Mb)	Memory Usage (MaxW-FSM) (Mb)
Mico	550000	733.18	786.80
	450000	794.58	811.76
	245000	5796	X
FreeAssoc	89000	118.23	200.73
	88500	118.23	5231
	15650	2153	X

in Table 4.3 mean that the algorithm never terminated its execution and thus no memory reading could be taken. As expected, WeFreS requires much less memory than when compared to the baseline approach MaxW-FSM, thus outperforming it in terms of memory consumption as well.

4.3.1.2 Comparison with ReSuM

As discussed in section 2.4.2, if all the edge weights in a given graph have values in the closed interval $[0, 1]$, ReSuM can be used to determine subgraphs whose weighted support satisfy a given user-defined threshold. In this section, we used the Amazon graph dataset, which has its weights normalized, to compare the performance of WeFreS with ReSuM when the weights are within the interval $[0, 1]$.

It should be noted that, the functionality of the baseline MaxW-DoubleCSP algorithm introduced in the preceding section is the same as the working principle of ReSuM in a normalized dataset. Since the maximum possible weight in a normalized dataset is 1, MaxW-DoubleCSP would include all subgraphs S for whom $MNI(S) \times maxW \geq \tau \implies MNI(S) \times 1 \geq \tau \implies MNI(S) \geq \tau$, which is equivalent to running GraMi first to find all frequent subgraphs. MaxW-Double CSP then ascertains which subgraphs from this list of frequent graphs are weighted frequent, an action that is also done by *ReSuM*. As such, we do not explicitly include the experimental results of MaxW-DoubleCSP in this section, but it should be implied

FIGURE 4.4: Runtime Comparison with *ReSuM* in the *Amazon* Dataset

that the results gained from experimental results on *ReSuM* shall be identical to the results gained from it.

Figure 4.4 shows the runtime comparisons of *ReSuM*, *MaxW-FSM* and *WeFreS*. While the difference in runtime remains relatively low in high threshold values, as soon as the threshold value drops to low values, leading to a greater

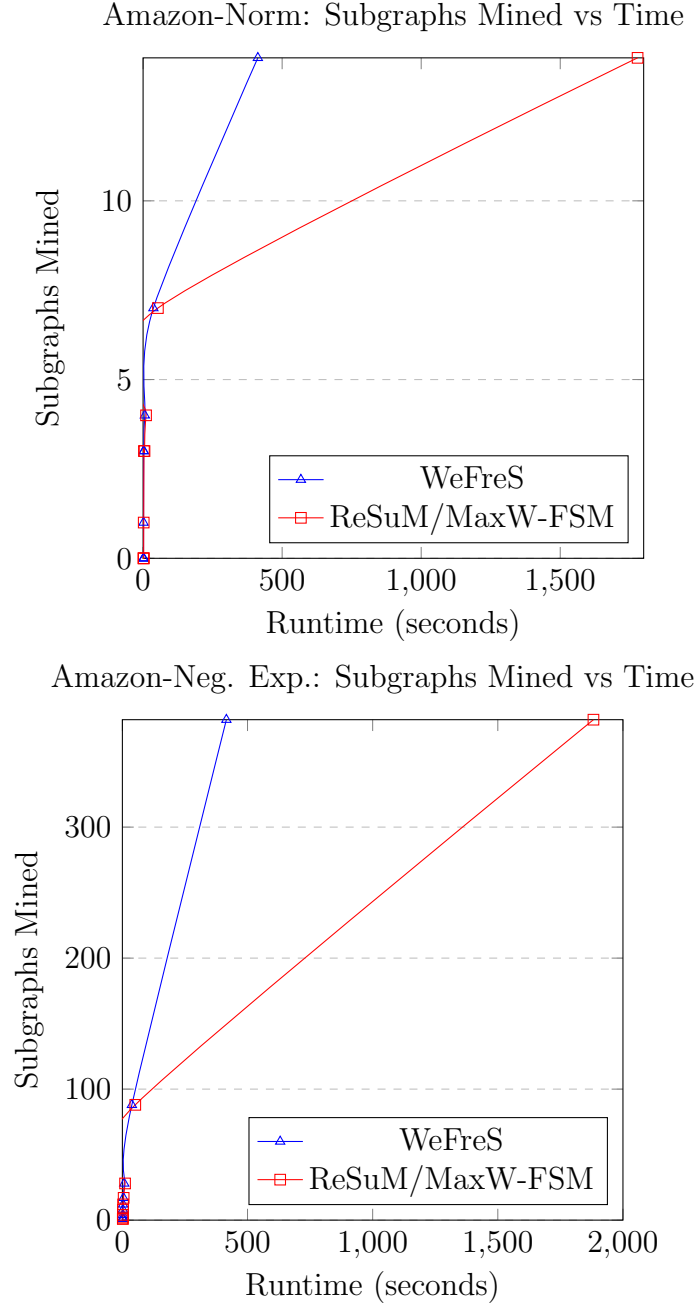
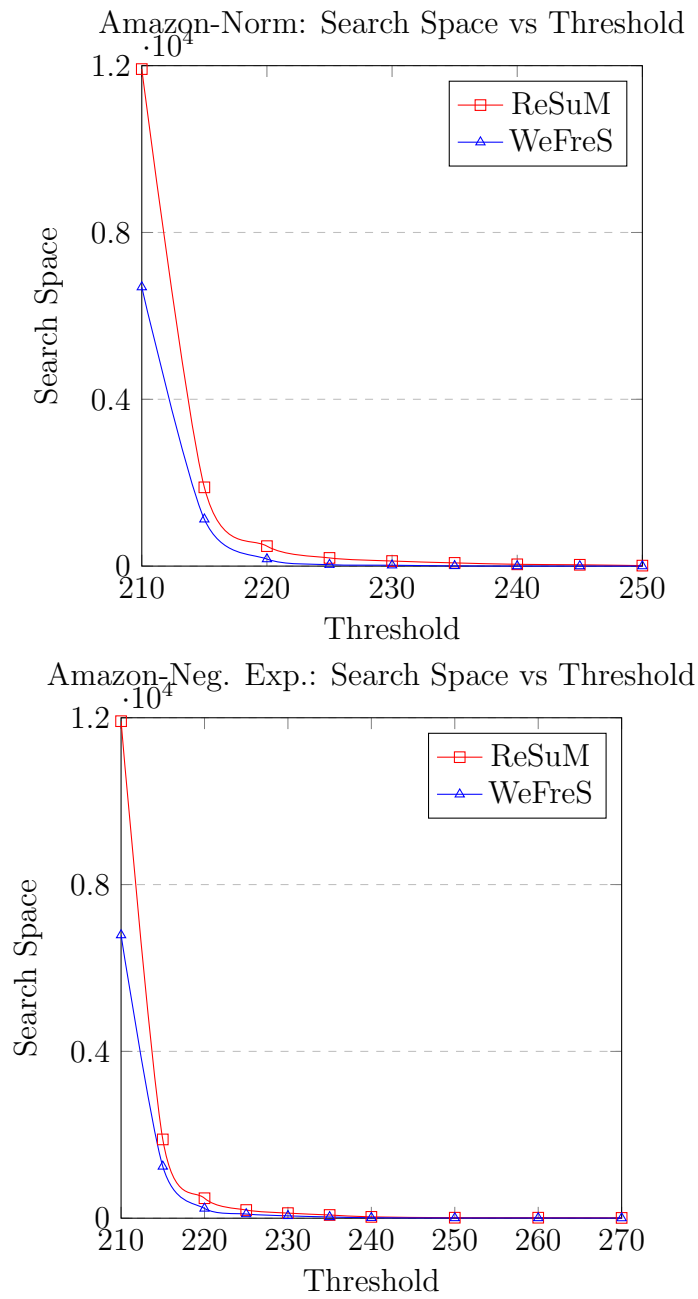


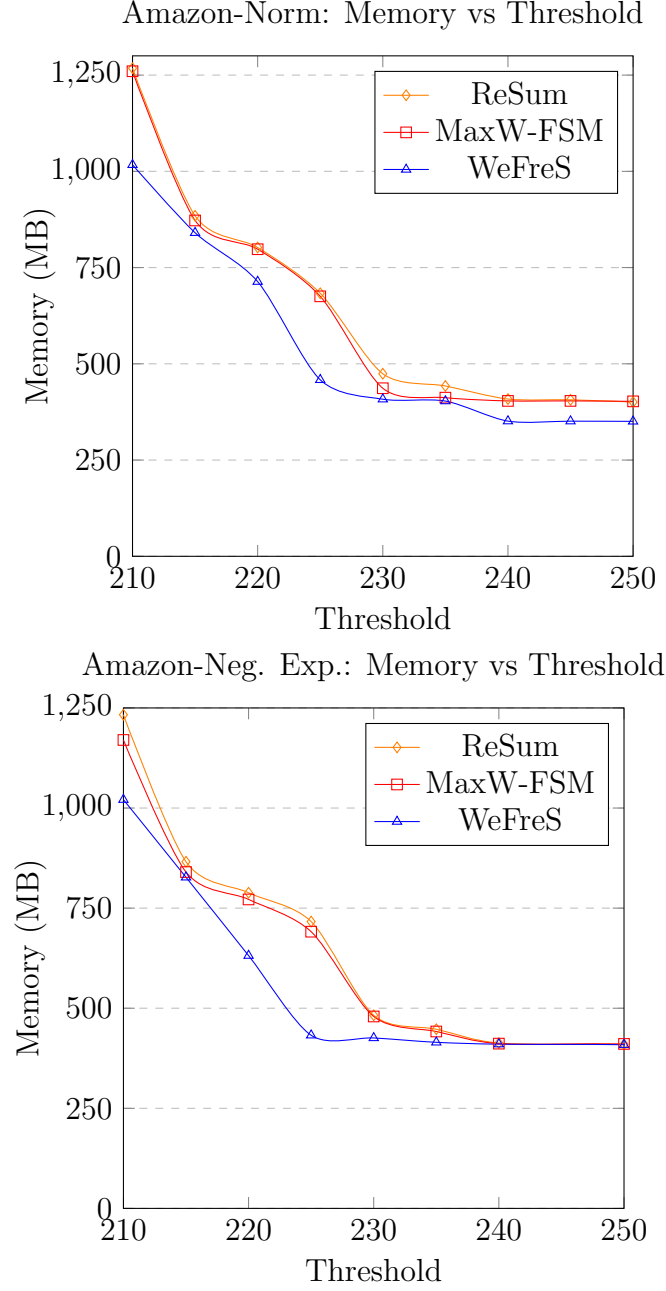
FIGURE 4.5: Number of Subgraphs Mined in a Given Time Constraint (*Amazon* Dataset)

number of subgraphs being mined, WeFreS works more than four times faster than the existing approach ReSuM. Figure 4.5 also demonstrates the superiority of WeFreS over ReSuM in terms of the number of subgraphs mined by an approach in a given time constraint.

The reduction of runtime can again be attributed to the reduction of search space, along with the novel CSP model we have proposed for weighted frequent subgraph mining from single large graphs. Search space comparisons are demonstrated in Figure 4.6.

Finally Figure 4.7 shows comparisons in memory usage in between the two approaches. WeFreS again requires less memory than ReSuM and MaxW-FSM, mainly due to the advantages of the reduced search space.

FIGURE 4.6: Search Space Comparison with *ReSuM* in the *Amazon* Dataset

FIGURE 4.7: Memory Usage Comparison with *ReSuM* in the *Amazon* Dataset

In summary, experimental results on the Amazon dataset in both statistical weight distributions strongly suggest that WeFreS offers an improved approach to mining frequent subgraphs in weighted single large graphs even when all edge-weights are in the real-valued interval $[0, 1]$.

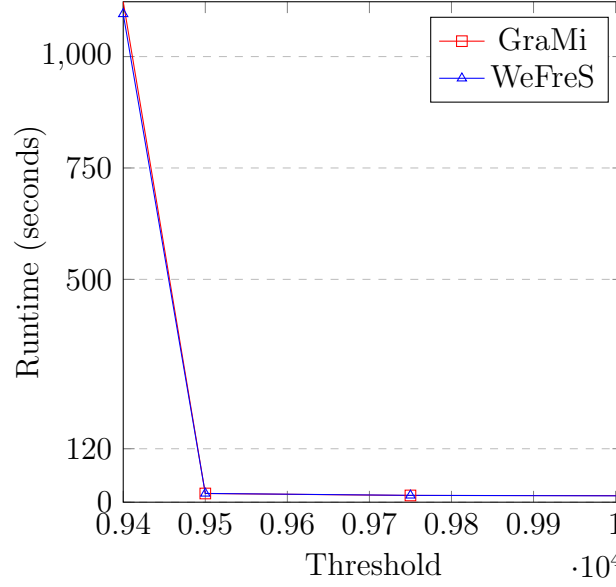
4.3.2 Comparison with Unweighted Frequent Subgraph Mining Approaches

In order to prove that WeFreS offers a truly general solution to the problem of frequent subgraph mining from single large graphs, and can perform sufficiently well even in the absence of edge-weights, we compare the performance of WeFreS with GraMi, a state-of-the-art approach for mining frequent subgraphs from unweighted single large graphs, using the unweighted versions of all three databases. In this section, all edge-weights in all three graphs were assigned the value 1.

In such cases, the segment tree we introduce in section 3.2.2.3 is compressed to having a single node only, whose weight sum and count is equal to the number of edges in the input graph. As such, the mechanisms used to determine *MaxPosW* do not induce much overhead in the mining process. A slight memory and runtime overhead is induced by the use of the *statusFlag* in our proposed CSP model, and due to it not being used in the CSP model adapted by GraMi.

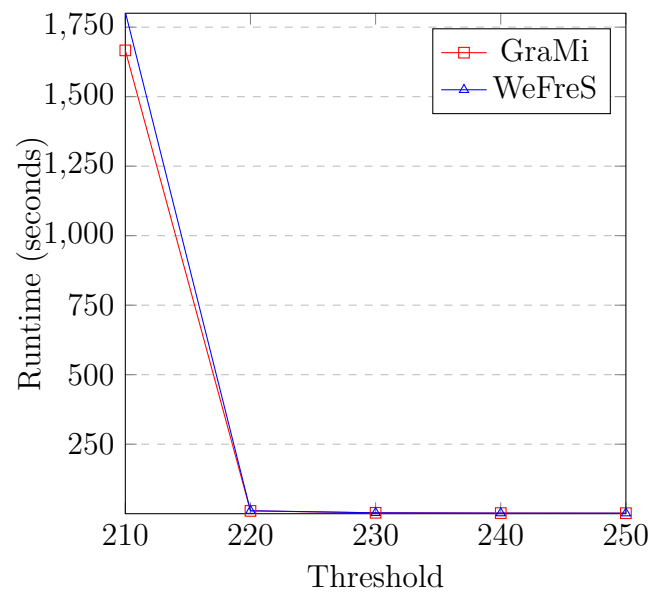
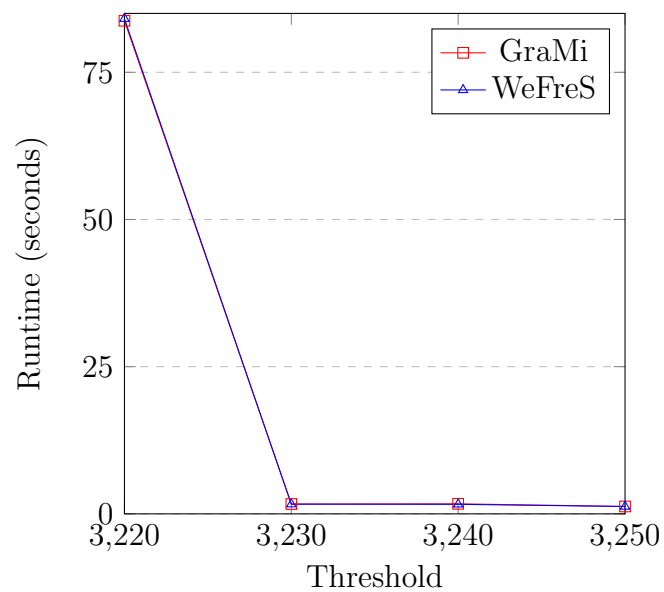
The main functional difference in between these two approaches is in the nature of the pre-pruning and optimization measures taken at the start of the mining process. While both WeFreS and GraMi delete infertile, and in this case infrequent, nodes and edges, WeFreS takes the additional step to prune out nodes and not consider node labels that have no frequent edges adjacent to them.

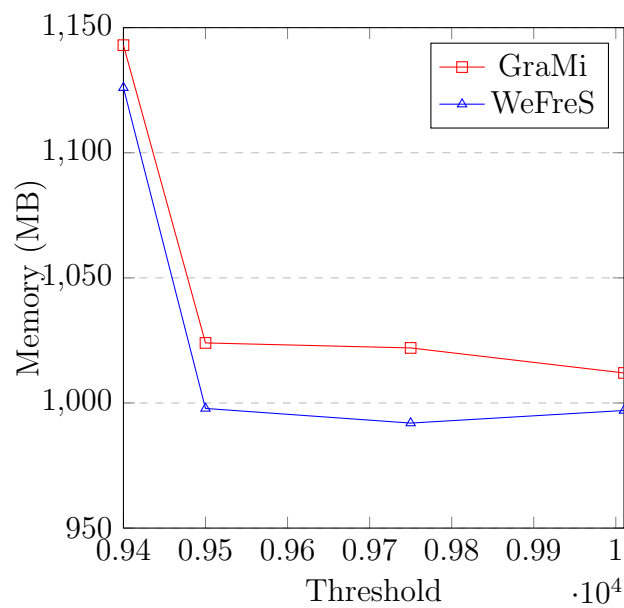
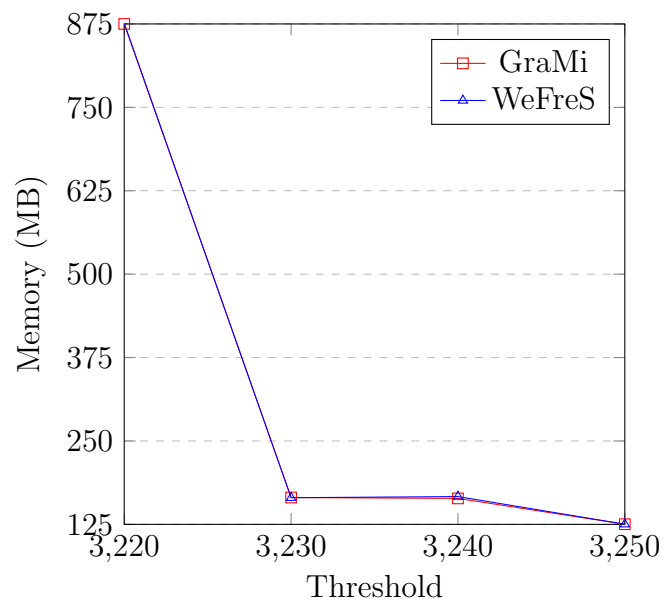
The second difference takes place due to the differing principles used while relabelling the node labels. Being a classical frequent subgraph mining approach that never take weights into considerations, GraMi sorts the node labels in decreasing order of frequency and relabels them accordingly. WeFreS, being designed to take weights into consideration by default, sorts the node labels in decreasing order of the sum of the weights of adjacent edges. In this context, WeFreS sorts the node labels in decreasing order of number of adjacent edges.

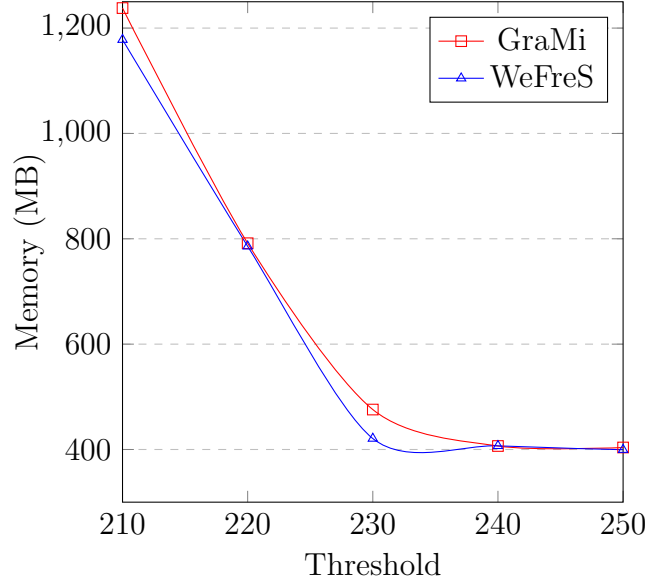
FIGURE 4.8: *MiCo*-unweighted: Runtime Comparison with *GraMi*

While this presents WeFreS with a slight disadvantage while mining frequent subgraphs, the difference induced in the runtimes of the two algorithms is negligible in all three datasets, as shown in Figures 4.8, 4.9 and 4.10. In most cases, the readings virtually overlap, showing how close the runtimes of these approaches really are.

The additional measure of deleting node labels that have no frequent edges adjacent to them actually helps reduce the search space to a small extent, and leads to an overall reduced memory consumption of the mining process in WeFreS, when compared to GraMi. The observed memory usage of both approaches are shown in Figures 4.11, 4.12 and 4.13. In summary, the results in this section show that even though WeFreS is primarily designed for mining weighted frequent subgraphs from weighted single large graphs, it can be used in unweighted setups as well. Experimental results strongly suggest that the performance of WeFreS is comparable to GraMi in unweighted setups, and can actually lead to less memory usage.

FIGURE 4.9: *Amazon-unweighted*: Runtime Comparison with *GraMi*FIGURE 4.10: *FreeAssoc-unweighted*: Runtime Comparison with *GraMi*

FIGURE 4.11: *MiCo*-unweighted: Memory Usage Comparison with *GraMi*FIGURE 4.12: *FreeAssoc*-unweighted: Memory Usage Comparison with *GraMi*

FIGURE 4.13: *Amazon-unweighted*: Memory Comparison with *GraMi*

4.4 Feasibility Analysis

The experimental analysis conducted on real-life data shows that our algorithm achieves significant improvement over all existing methods. The complexity analysis provided in section 3.3 indicates the runtime exponentially increases with the size of the subgraph. Generally, the size of frequent patterns (subgraph in this case) in real-world datasets is significantly smaller compared to its entire size. So, the number of nodes in the subgraph in question is usually very low resulting in subgraphs with negligible size. The introduction of *MaxPosW* prevents the generation of unnecessary subgraphs with large number of nodes. Other methods either use *MaxW* or a ‘score’ function like *ReSuM* to extend the subgraph as long as the edges with high significance have not been used. So, in other existing methods, subgraphs generated have greater number of nodes resulting in a notable increase in runtime and memory. For example, in the *FreeAssoc* dataset, for threshold 88500, there are 0 frequent subgraphs. The *MaxPosW* measure in *WeFreS* successfully prunes everything and avoids subgraph extensions. But *ReSum* extends to 551 subgraphs. With lower thresholds like 15650, *WeFreS* mines 22 weighted

frequent subgraphs by searching 899 subgraphs with trivial number of nodes. *ReSum* advances to extending subgraphs to a substantial depth and after running for hours terminates due to memory overflow.

As shown in our experimental results, *WeFreS* avoids generating large constraint graphs and proves to be scalable. Thus, we can conclude that *WeFreS* is feasible for use in real-world applications.

Chapter 5

Conclusions

Graphs are an important tool that can be used to model complex relationships among different types of entities in various real-world applications. Mining large graph datasets can lead us to discovering new information in various fields of our life. The addition of edge-weights can add even more representational power to graphs, allowing users to specify the relative importance of different interactions. Here, we have focused on mining weighted frequent subgraphs from single large graphs, and in doing so, we have proposed **WeFreS**, an efficient weighted subgraph mining algorithm that mines subgraphs whose weighted support is greater than or equal to a given user-defined threshold.

5.1 Research Summary

Experimental results on real life datasets strongly indicate that the prunings introduced in **WeFreS** make it hugely faster than other existing weighted single large graph mining methods. The fact that **WeFreS** considers weighted support as the product of the frequency support of a subgraph with the average of the weights of the edges present in it, can help us mine patterns that are both regular and significant. Such patterns can reveal useful information about the graph data that exists around us, and can contribute to enhancing our knowledge in a variety of

real-life scenarios including analyzing social networks, chemical networks, software bugs, natural language texts and a lot more. **WeFreS** also introduces a novel approach to determining *MaxPosW*, the maximum possible weight attainable after extension, an approach that can be adopted in other forms of weighted pattern mining as well.

5.2 Future Research Scopes

The proposed approach, **WeFreS** has been proved by experimental analysis to be both efficient and conservative in memory consumption. However, the research still has potential to be extended further, and the concepts introduced in Chapter 3 can be modified to be put into use in other forms of weighted pattern mining as well. Scopes of improvement and extension of our existing work, and the implications it can have on future research can be summarized as follows:

- Future research can be done on using the *MaxPosW*-approach introduced here and applying it in other fields of data mining such as weighted sequential mining, weighted frequent itemset mining among others.
- We can explore ways to further tighten the *MaxPosW* upper bound by incorporating and taking advantage of the features of the input graph.
- Initial analysis shows that using ternary search for determining the value of *MaxPosW* is not always necessary, and in certain cases, the *MaxPosW* of a subgraph can be determined in an $O(1)$ approach. Further investigation on this property can lead us to designing an improved version of our currently proposed approach.

-
- Developing an approximate approach for mining weighted frequent subgraphs from single large graphs, which trades off a certain degree of accuracy for further efficiency.

Bibliography

- [1] Yun, Unil, and John J. Leggett. "WFIM: weighted frequent itemset mining with a weight range and a minimum weight." Proceedings of the 2005 SIAM international conference on data mining. Society for Industrial and Applied Mathematics, 2005.
- [2] S. Fakhraei, J. Foulds, M. Shashanka, and L. Getoor, 'Collective spammer detection in evolving multi-relational social networks', in Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, 2015, pp. 1769-1778.
- [3] Ahmed, C. F., Tanbeer, S. K., Jeong, B., Lee, Y., & Choi, H. (2012). Single-pass Incremental and Interactive Mining for Weighted Frequent Patterns. *Expert Systems with Applications*, 39(9), 7976-7994. doi:10.1016/j.eswa.2012.01.117
- [4] Han, Jiawei, Micheline Kamber, and Jian Pei. "Data mining: concepts and techniques (the Morgan Kaufmann Series in data management systems)." Morgan Kaufmann (2000).
- [5] Agrawal, Rakesh, and Ramakrishnan Srikant. "Fast algorithms for mining association rules." Proc. 20th int. conf. very large data bases, VLDB. Vol. 1215. 1994.
- [6] JIAWEI HAN, JIAN PEI, YIWEN YIN, RUNYING MAO. "Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach"

-
- [7] Yan, Xifeng, and Jiawei Han. "gspan: Graph-based substructure pattern mining." *Data Mining, 2002. ICDM 2003. Proceedings. 2002 IEEE International Conference on*. IEEE, 2002.
- [8] Elseidy, Mohammed, et al. "Grami: Frequent subgraph and pattern mining in a single large graph." *Proceedings of the VLDB Endowment* 7.7 (2014): 517-528.
- [9] Bringmann, Björn, and Siegfried Nijssen. "What is frequent in a single graph?." *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, Berlin, Heidelberg, 2008.
- [10] Md. Ashraful Islam, Chowdhury Farhan Ahmed, Carson K. Leung & Calvin S.H. Hoi "WFSM-MaxPWS: An Efficient Approach for Mining"
- [11] Yang, Jiong, et al. "WIGM: discovery of subgraph patterns in a large weighted graph." *Proceedings of the 2012 SIAM International Conference on Data Mining*. Society for Industrial and Applied Mathematics, 2012.
- [12] Preti, Giulia, et al. "Beyond Frequencies: Graph Pattern Mining in Multi-weighted Graphs." (2018).
- [13] Borgelt, Christian, and Michael R. Berthold. "Mining molecular fragments: Finding relevant substructures of molecules." *Data Mining, 2002. ICDM 2003. Proceedings. 2002 IEEE International Conference on*. IEEE, 2002.
- [14] Nowozin, Sebastian, et al. "Weighted substructure mining for image analysis." *Computer Vision and Pattern Recognition, 2007. CVPR'07. IEEE Conference on*. IEEE, 2007.]
- [15] Eichinger F., Bhm K., Huber M. (2008) Mining Edge-Weighted Call Graphs to Localise Software Bugs. In: Daelemans W., Goethals B., Morik K. (eds) *Machine Learning and Knowledge Discovery in Databases. ECML PKDD 2008*. Lecture Notes in Computer Science, vol 5211. Springer, Berlin, Heidelberg

- [16] Jiang C., Coenen F., Sanderson R., Zito M. (2010) Text Classification using Graph Mining-based Feature Extraction. In: Bramer M., Ellis R., Petridis M. (eds) Research and Development in Intelligent Systems XXVI. Springer, London
- [17] Murata, Tsuyoshi, and Sakiko Moriyasu. "Link prediction of social networks based on weighted proximity measures." Proceedings of the IEEE/WIC/ACM international conference on web intelligence. IEEE Computer Society, 2007.
- [18] Liakos, Panagiotis, and Katia Papakonstantinou. "On the Impact of Social Cost in Opinion Dynamics." ICWSM. 2016.
- [19] Ata, Sezin Kircali, et al. "Disease Gene Classification with Metagraph Representations." Data Mining for Systems Biology. Humana Press, New York, NY, 2018. 211-224.
- [20] Pinto, Joo Carlos Costa. "Exploiting JavaScript Birthmarking Techniques for Code Theft Detection." (2016).
- [21] Social Spammers in Evolving Multi-Relational Social Network Dataset: <https://lincs-data.soe.ucsc.edu/public/socialspammer/>
- [22] Jiang, Chuntao, Frans Coenen, and Michele Zito. "Frequent sub-graph mining on edge weighted graphs." In International Conference on Data Warehousing and Knowledge Discovery, pp. 77-88. Springer, Berlin, Heidelberg, 2010.
- [23] Lincs statistical research group: <https://lincs.soe.ucsc.edu/data>
- [24] Arizona State University's Social Computing Data Repository: <http://socialcomputing.asu.edu>
- [25] Stanford Network Analysis Project: <https://snap.stanford.edu/>
- [26] Han, Jiawei, et al. "Frequent pattern mining: current status and future directions." Data Mining and Knowledge Discovery 15.1 (2007): 55-86.

- [27] Fayyad, Usama M., et al. "Advances in knowledge discovery and data mining." (1996).
- [28] Yiu, Man Lung, and Nikos Mamoulis. "Frequent-pattern based iterative projected clustering." *Data Mining, 2003. ICDM 2003. Third IEEE International Conference on.* IEEE, 2003.
- [29] Cheng, Hong, et al. "Discriminative frequent pattern analysis for effective classification." *2007 IEEE 23rd International Conference on Data Engineering.* IEEE, 2007.
- [30] Hariri, Negar, Bamshad Mobasher, and Robin Burke. "Context-aware music recommendation based on latenttopic sequential patterns." *Proceedings of the sixth ACM conference on Recommender systems.* ACM, 2012.
- [31] Ghorashi, Seyed Hamid, et al. "A frequent pattern mining algorithm for feature extraction of customer reviews." *International Journal of Computer Science Issues (IJCSI)* 9.4 (2012): 29.
- [32] He, Zengyou, et al. "FP-outlier: Frequent pattern based outlier detection." *Computer Science and Information Systems* 2.1 (2005): 103-118.
- [33] S. Kumar, F. Spezzano, V.S. Subrahmanian, C. Faloutsos. Edge Weight Prediction in Weighted Signed Networks. *IEEE International Conference on Data Mining (ICDM)*, 2016.
- [34] Stark, Chris, et al. "BioGRID: a general repository for interaction datasets." *Nucleic acids research* 34.suppl_1(2006) : D535 – D539. *Lu, Qing, and Lise Getoor.* "Link – based classification." *Proceedings of the 20th International Conference on Machine Learning* (03).2003.
- [35][36] Akoglu, Leman, Mary McGlohon, and Christos Faloutsos. "Oddball: Spotting anomalies in weighted graphs." *Pacific-Asia Conference on Knowledge Discovery and Data Mining.* Springer, Berlin, Heidelberg, 2010.

-
- [37] Islam, M.A., Ahmed, C.F., Leung, C.K., Hoi, C.S.: WFSM-MaxPWS: An efficient approach for mining weighted frequent subgraphs from edge-weighted graph databases. In: PAKDD, Springer (2018) 664676
 - [38] Babu, N., John, A.: A distributed approach to weighted frequent subgraph mining. In: ICETT, IEEE (2016) 17
 - [39] Ruining He and Julian McAuley. 2016. Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In WWW. 507517.
 - [40] Nelson, Douglas L., Cathy L. McEvoy, and Thomas A. Schreiber. "The University of South Florida free association, rhyme, and word fragment norms." Behavior Research Methods, Instruments, Computers 36.3 (2004): 402-407.

List of Notations

KDD - Knowledge Discovery from Data

WFIM - Weighted Frequent Itemset Mining

CSP - Constraint Satisfaction Problem

MNI - Minimum Image Based Support

τ - User-defined Minimum Weighted Support Threshold

MaxPosW - Maximum Possible Weight of a Subgraph

MaxW - Maximum Weight

MinW - Minimum Weight

min_sup - Minimum Support

WS - Weighted Support

sW - Weight of a Subgraph

sG - Support of a Subgraph

DFS - Depth First Search

cur_sum - Summation of the Weights of the Edges in the Subgraph

sum_i - Summation of the Weights of Highest Weighted *i* Edges among the Remaining Edges

h_i - Maximum Possible Average Weight a Subgraph Can Attain after Being Extended by *i* Edges

rem - Remaining Number of Edges

Avg - Average Weight

reqFreq - Required MNI for being Frequent

reqExt - Required MNI for having a Frequent Extension

maxMNI - Maximum MNI

minMNI - Minimum MNI

currentWeightofSubgraph - Current Weight of the Subgraph

statusFlag - Status Flag